

ola-ensemble-learning

July 26, 2025

1 Problem Statement

Ola is facing a high driver churn rate, increasing hiring costs and affecting operational efficiency. The goal is to predict whether a driver will leave (attrition = 1) or stay (attrition = 0) based on demographic, performance, and tenure-related features.

Operational Challenges

Constant need to recruit and onboard new drivers, leading to increased hiring costs and logistical challenges.

High turnover disrupts supply-demand balance, especially during peak hours or in high-demand regions.

Retention Issues

Difficulty in retaining experienced drivers due to factors such as:

Income fluctuations

Better incentives from competitors like Uber

Job dissatisfaction or lack of long-term benefits

Business Impact

Frequent exits affect driver morale, customer satisfaction, and service reliability.

Acquiring new drivers is significantly more expensive than retaining current ones.

Attrition undermines operational efficiency and long-term scalability.

Objective

Develop a predictive model to classify whether a driver is likely to leave (attrition = 1) or stay (attrition = 0) with Ola, using historical data related to:

Demographics: Age, gender, city, etc.

Tenure information: Joining date, last working date

Performance metrics: Ratings, number of rides, income, driver grade, etc.

2 Column Profiling:

MMMM-YY : Reporting Date (Monthly)

Driver_ID : Unique id for drivers

Age : Age of the driver

Gender : Gender of the driver - Male : 0, Female: 1

City : City Code of the driver

Education_Level : Education level - 0 for 10+ ,1 for 12+ ,2 for graduate

Income : Monthly average Income of the driver

Date Of Joining : Joining date for the driver

LastWorkingDate : Last date of working for the driver

Joining Designation : Designation of the driver at the time of joining

Grade : Grade of the driver at the time of reporting

Total Business Value : The total business value acquired by the driver in a month (negative business indicates cancellation/refund or car EMI adjustments)

Quarterly Rating : Quarterly rating of the driver: 1,2,3,4,5 (higher is better)

3 Concepts Tested:

Ensemble Learning- Bagging

Ensemble Learning- Boosting

KNN Imputation of Missing Values

Working with an imbalanced dataset

```
[338]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```

from sklearn.preprocessing import MinMaxScaler

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, \
    classification_report
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.metrics import average_precision_score

from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
import datetime as dt

import warnings
warnings.filterwarnings('ignore')

```

```

[236]: data = pd.read_csv('ola_driver_scaler.csv')
data.head()

```

```

[236]:
   Unnamed: 0  MMM-YY  Driver_ID  Age  Gender  City  Education_Level  \
0           0  01/01/19          1  28.0    0.0  C23                2
1           1  02/01/19          1  28.0    0.0  C23                2
2           2  03/01/19          1  28.0    0.0  C23                2
3           3  11/01/20          2  31.0    0.0  C7                 2
4           4  12/01/20          2  31.0    0.0  C7                 2

   Income  Dateofjoining  LastWorkingDate  Joining  Designation  Grade  \
0   57387      24/12/18              NaN          1          1
1   57387      24/12/18              NaN          1          1
2   57387      24/12/18      03/11/19          1          1
3   67016      11/06/20              NaN          2          2
4   67016      11/06/20              NaN          2          2

   Total Business Value  Quarterly Rating
0           2381060          2
1           -665480          2
2              0          2

```

3	0	1
4	0	1

Exploring the data

```
[237]: # Checking the number of rows and columns
print(f"The number of rows: {data.shape[0]:,} \nThe number of columns: {data.
↪shape[1]:,}")
```

The number of rows: 19,104
The number of columns: 14

```
[238]: # Check all column names
data.columns
```

```
[238]: Index(['Unnamed: 0', 'MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City',
            'Education_Level', 'Income', 'Dateofjoining', 'LastWorkingDate',
            'Joining Designation', 'Grade', 'Total Business Value',
            'Quarterly Rating'],
            dtype='object')
```

Observations on Data

```
[239]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19104 entries, 0 to 19103
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            19104 non-null  int64
1   MMM-YY                19104 non-null  object
2   Driver_ID             19104 non-null  int64
3   Age                   19043 non-null  float64
4   Gender                19052 non-null  float64
5   City                  19104 non-null  object
6   Education_Level       19104 non-null  int64
7   Income                19104 non-null  int64
8   Dateofjoining         19104 non-null  object
9   LastWorkingDate       1616 non-null   object
10  Joining Designation    19104 non-null  int64
11  Grade                 19104 non-null  int64
12  Total Business Value  19104 non-null  int64
13  Quarterly Rating      19104 non-null  int64
dtypes: float64(2), int64(8), object(4)
memory usage: 2.0+ MB
```

Data Observation

Total Records: 19,104 rows

Total Features: 14 columns

Missing Values:

Age: 61 missing entries

Gender: 52 missing entries

LastWorkingDate: only 1,616 non-null → strong candidate for creating the target variable attrition

```
[240]: data.describe()
```

```
[240]:
```

	Unnamed: 0	Driver_ID	Age	Gender	\
count	19104.000000	19104.000000	19043.000000	19052.000000	
mean	9551.500000	1415.591133	34.668435	0.418749	
std	5514.994107	810.705321	6.257912	0.493367	
min	0.000000	1.000000	21.000000	0.000000	
25%	4775.750000	710.000000	30.000000	0.000000	
50%	9551.500000	1417.000000	34.000000	0.000000	
75%	14327.250000	2137.000000	39.000000	1.000000	
max	19103.000000	2788.000000	58.000000	1.000000	

	Education_Level	Income	Joining	Designation	Grade	\
count	19104.000000	19104.000000		19104.000000	19104.000000	
mean	1.021671	65652.025126		1.690536	2.252670	
std	0.800167	30914.515344		0.836984	1.026512	
min	0.000000	10747.000000		1.000000	1.000000	
25%	0.000000	42383.000000		1.000000	1.000000	
50%	1.000000	60087.000000		1.000000	2.000000	
75%	2.000000	83969.000000		2.000000	3.000000	
max	2.000000	188418.000000		5.000000	5.000000	

	Total Business Value	Quarterly Rating
count	1.910400e+04	19104.000000
mean	5.716621e+05	2.008899
std	1.128312e+06	1.009832
min	-6.000000e+06	1.000000
25%	0.000000e+00	1.000000
50%	2.500000e+05	2.000000
75%	6.997000e+05	3.000000
max	3.374772e+07	4.000000

```
[241]: data.drop(columns='Unnamed: 0', axis=1, inplace=True, errors='ignore')
```

```
[242]: data.nunique()
```

```
[242]: MMM-YY          24
      Driver_ID       2381
      Age             36
      Gender          2
      City            29
      Education_Level  3
      Income          2383
      Dateofjoining   869
      LastWorkingDate 493
      Joining Designation 5
      Grade           5
      Total Business Value 10181
      Quarterly Rating 4
      dtype: int64
```

```
[243]: data.isna().sum()
```

```
[243]: MMM-YY          0
      Driver_ID         0
      Age             61
      Gender          52
      City            0
      Education_Level  0
      Income           0
      Dateofjoining    0
      LastWorkingDate 17488
      Joining Designation 0
      Grade           0
      Total Business Value 0
      Quarterly Rating 0
      dtype: int64
```

```
[244]: data.duplicated().value_counts()
```

```
[244]: False      19104
      Name: count, dtype: int64
```

```
[245]: data1 = df.copy(deep=True)
```

Categorical Data Cleaning and Encoding

```
[246]: categorical_columns = ['Gender', 'City', 'Education_Level', 'Joining_
      ↪Designation', 'Grade']
      data[categorical_columns] = data[categorical_columns].astype('category')
      data['Gender'].replace({0.0: 'Male', 1.0: 'Female'}, inplace=True)
      data['Education_Level'].replace({0: '10+', 1: '12+', 2: 'Graduate'}, inplace=True)
```

```
[247]: data.columns
```

```
[247]: Index(['MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',  
        'Income', 'Dateofjoining', 'LastWorkingDate', 'Joining Designation',  
        'Grade', 'Total Business Value', 'Quarterly Rating'],  
        dtype='object')
```

Date Formatting and Extraction

```
[248]: # Convert ReportingMonthYear to datetime, handling both 'MMM-YY' and  
        ↪ 'ReportingMonthYear' scenarios  
        if 'MMM-YY' in data.columns:  
            data.rename(columns={'MMM-YY': 'ReportingMonthYear'}, inplace=True)  
  
        # Convert string to datetime  
        data['ReportingMonthYear'] = pd.to_datetime(data['ReportingMonthYear'],  
            ↪ errors='coerce')  
        data['Dateofjoining'] = pd.to_datetime(data['Dateofjoining'], format='%d/%m/  
            ↪ %y', errors='coerce')  
        data['LastWorkingDate'] = pd.to_datetime(data['LastWorkingDate'], format='%d/%m/  
            ↪ %y', errors='coerce')
```

Reporting Month and Year Features

```
[249]: data.rename(columns={'MMM-YY': 'ReportingMonthYear'}, inplace=True)  
        data['ReportingMonthYear'] = pd.to_datetime(data['ReportingMonthYear'],  
            ↪ format='%b-%y', errors='coerce')
```

Joining Date Features

```
[250]: data['Monthofjoining'] = data['Dateofjoining'].dt.month  
        data['Yearofjoining'] = data['Dateofjoining'].dt.year
```

Target Variable Construction — Churn

```
[251]: data['Churn'] = data.groupby('Driver_ID')['LastWorkingDate'].transform('last')  
        data['Churn'] = data['Churn'].apply(lambda x: 0 if pd.isnull(x) else 1)
```

```
[252]: data.head()
```

```
[252]:   ReportingMonthYear  Driver_ID  Age  Gender  City  Education_Level  Income  \  
0      2019-01-01         1  28.0   Male   C23      Graduate    57387  
1      2019-02-01         1  28.0   Male   C23      Graduate    57387  
2      2019-03-01         1  28.0   Male   C23      Graduate    57387  
3      2020-11-01         2  31.0   Male    C7      Graduate    67016  
4      2020-12-01         2  31.0   Male    C7      Graduate    67016
```

```
   Dateofjoining  LastWorkingDate  Joining Designation  Grade  \  

```

0	2018-12-24	NaT	1	1
1	2018-12-24	NaT	1	1
2	2018-12-24	2019-11-03	1	1
3	2020-06-11	NaT	2	2
4	2020-06-11	NaT	2	2

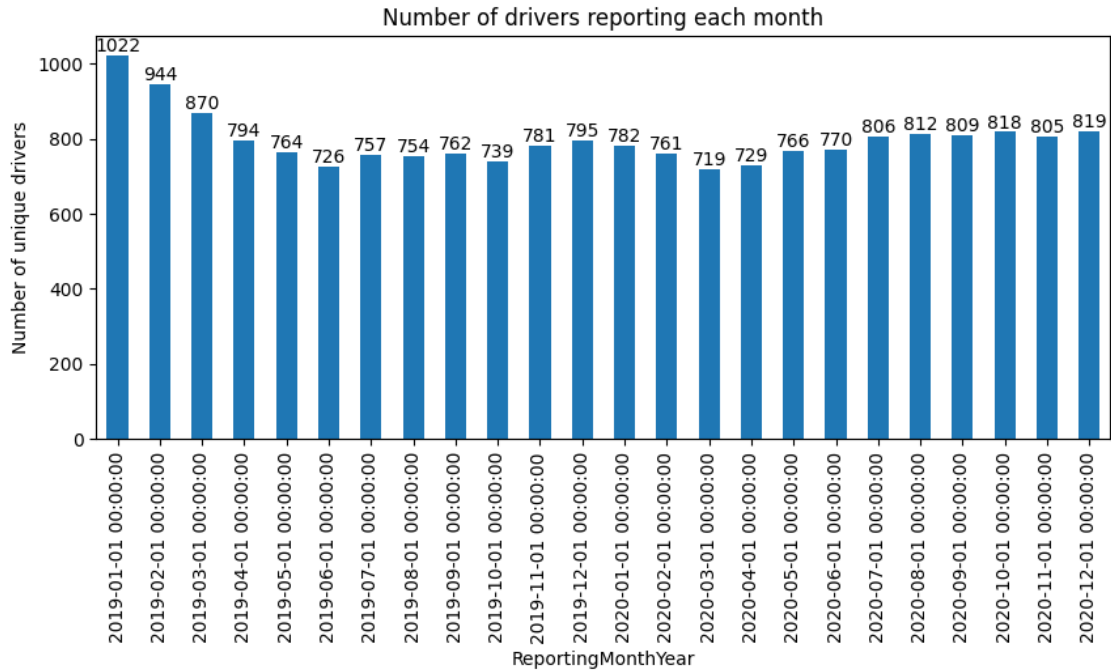
	Total Business Value	Quarterly Rating	Monthofjoining	Yearofjoining \
0	2381060	2	12	2018
1	-665480	2	12	2018
2	0	2	12	2018
3	0	1	6	2020
4	0	1	6	2020

	Churn
0	1
1	1
2	1
3	0
4	0

4 Exploratory Data Analysis

Univariate analysis

```
[253]: plt.figure(figsize=(10,4))
temp_df = data.groupby('ReportingMonthYear')['Driver_ID'].nunique()
ax = temp_df.plot(kind='bar')
ax.bar_label(ax.containers[0])
plt.ylabel('Number of unique drivers')
plt.title('Number of drivers reporting each month')
plt.show()
```

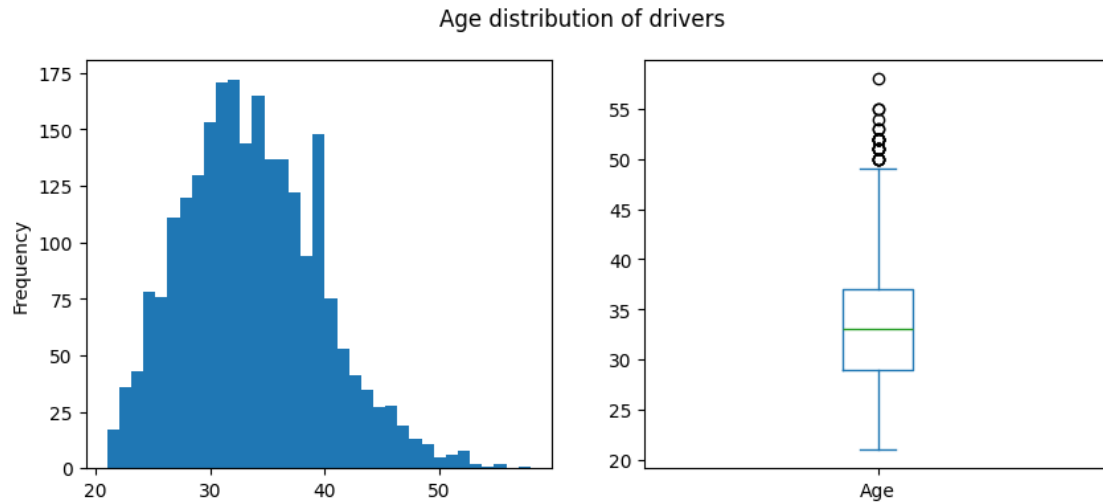



Insight

The month during which maximum number of drivers reported is January 2019. A total of 1022 drivers reported on January 2019

It then dropeed every month after January and has been stagnant at around 800 drivers reported every month

```
[254]: fig, axs = plt.subplots(1,2,figsize=(10,4))
temp_df = data.groupby('Driver_ID').agg({'Age':'last'})['Age']
temp_df.plot(ax=axs[0], kind='hist', bins=35)
temp_df.plot(ax=axs[1], kind='box')
fig.suptitle('Age distribution of drivers')
plt.show()
```



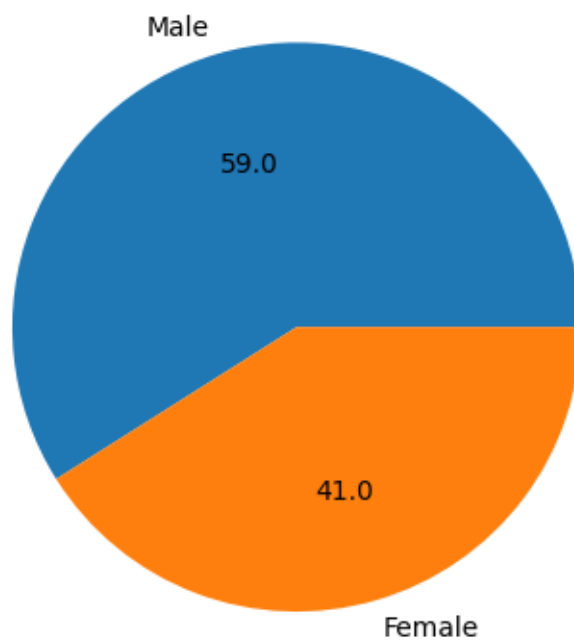
Insight

There are drivers from different age groups ranging from 21 to 58 years. Most of the drivers are in the age group of 30 to 35.

The distribution is mostly normal with little skewness towards the right.

```
[255]: temp_df = data.groupby('Driver_ID').agg({'Gender': 'first'})
temp_df['Gender'].value_counts().plot(kind='pie', autopct='%1f')
plt.title('Gender % distribution of drivers')
plt.ylabel('')
plt.show()
```

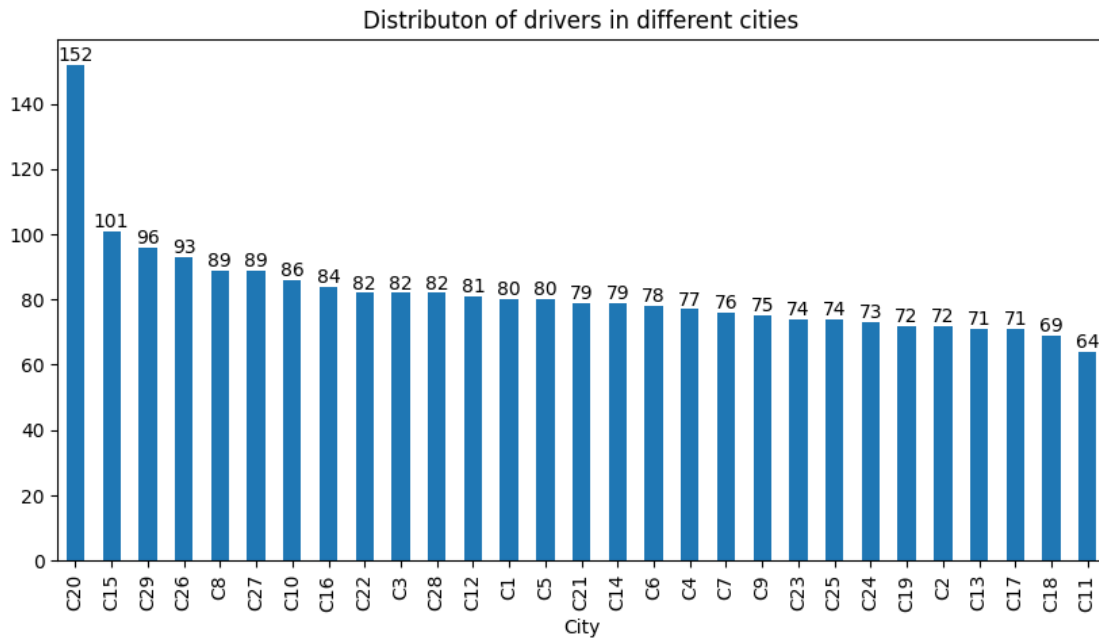
Gender % distribution of drivers



Insight

59% of the drivers are Male and remaining 41% are Female

```
[256]: plt.figure(figsize=(10,5))
temp_df = data.groupby('Driver_ID').agg({'City':'first'})
ax = temp_df['City'].value_counts().plot(kind='bar')
ax.bar_label(ax.containers[0])
plt.title('Distributon of drivers in different cities')
plt.show()
```

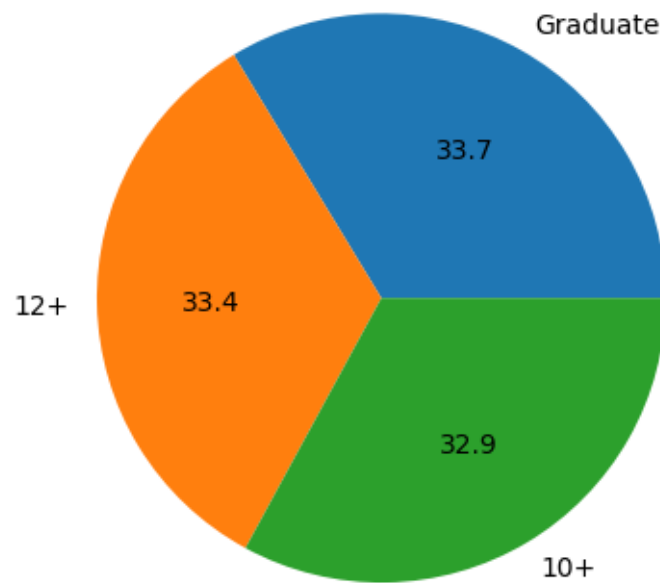


Insight

City C20 has the maximum number of drivers followed by city C15

```
[257]: temp_df = data.groupby('Driver_ID').agg({'Education_Level': 'first'})
temp_df['Education_Level'].value_counts().plot(kind='pie', autopct='%1f')
plt.ylabel('')
plt.title('Education level % distribution of drivers')
plt.show()
```

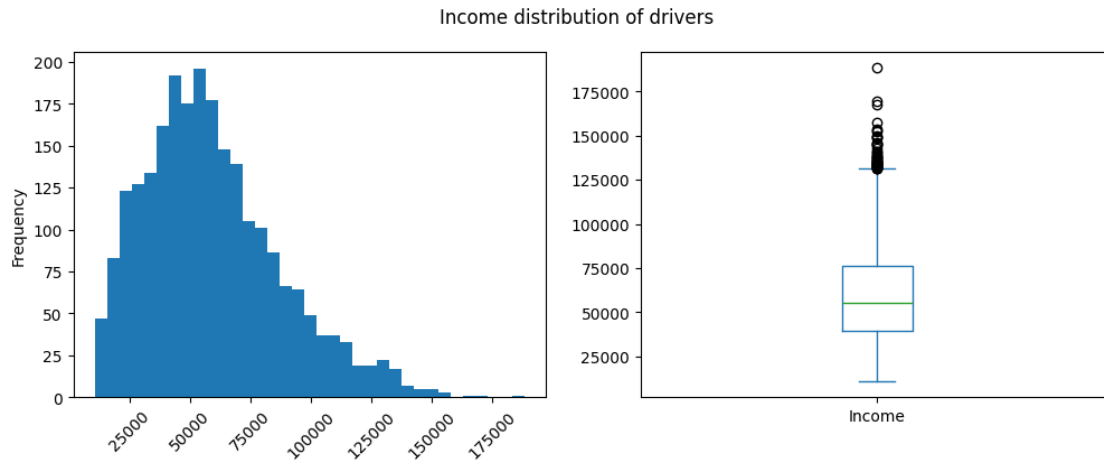
Education level % distribution of drivers



Insight

Almost equal proportion of drivers are from the 3 different education level

```
[258]: fig, axs = plt.subplots(1,2,figsize=(12,4))
temp_df = data.groupby('Driver_ID').agg({'Income':'last'})['Income']
temp_df.plot(ax=axs[0], kind='hist', bins=35, rot=45)
temp_df.plot(ax=axs[1], kind='box')
fig.suptitle('Income distribution of drivers')
plt.show()
```

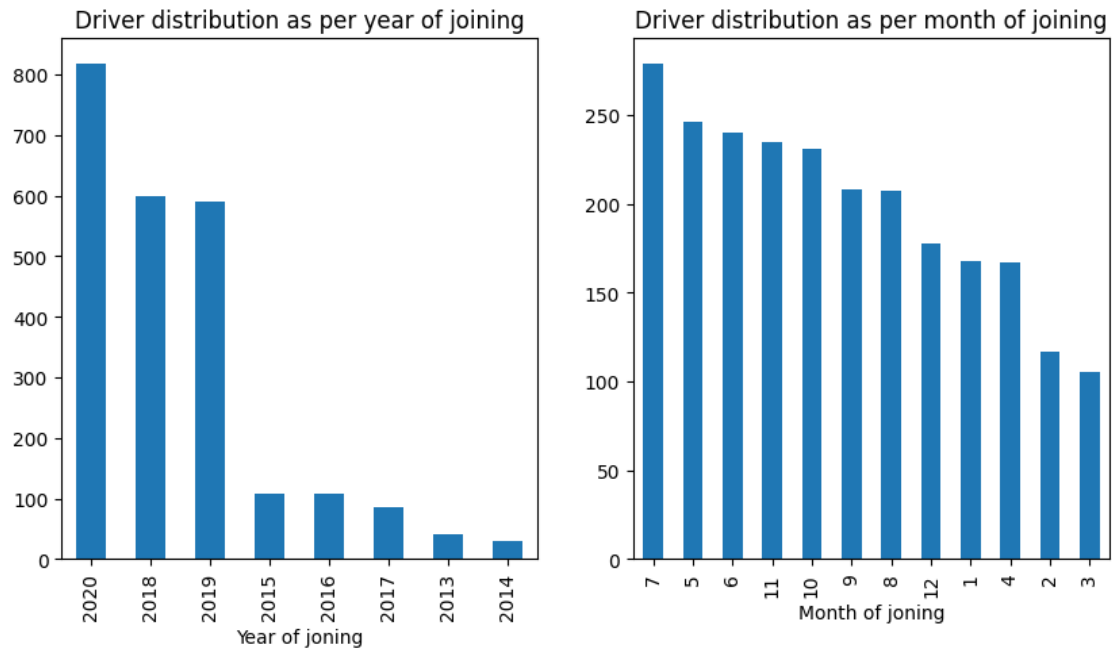


Insight

Most of the drivers have an average monthly income of 40k to 75k

The distribution is right skewed

```
[259]: temp_df = data.groupby('Driver_ID').agg({'Dateofjoining':
        ↳'first'})['Dateofjoining']
fig, axs = plt.subplots(1,2,figsize=(10,5))
temp_df.dt.year.value_counts().plot(kind='bar', ax=axs[0], xlabel='Year of_
        ↳joining', title='Driver distribution as per year of joining')
temp_df.dt.month.value_counts().plot(kind='bar', ax=axs[1], xlabel='Month of_
        ↳joining', title='Driver distribution as per month of joining')
plt.show()
```

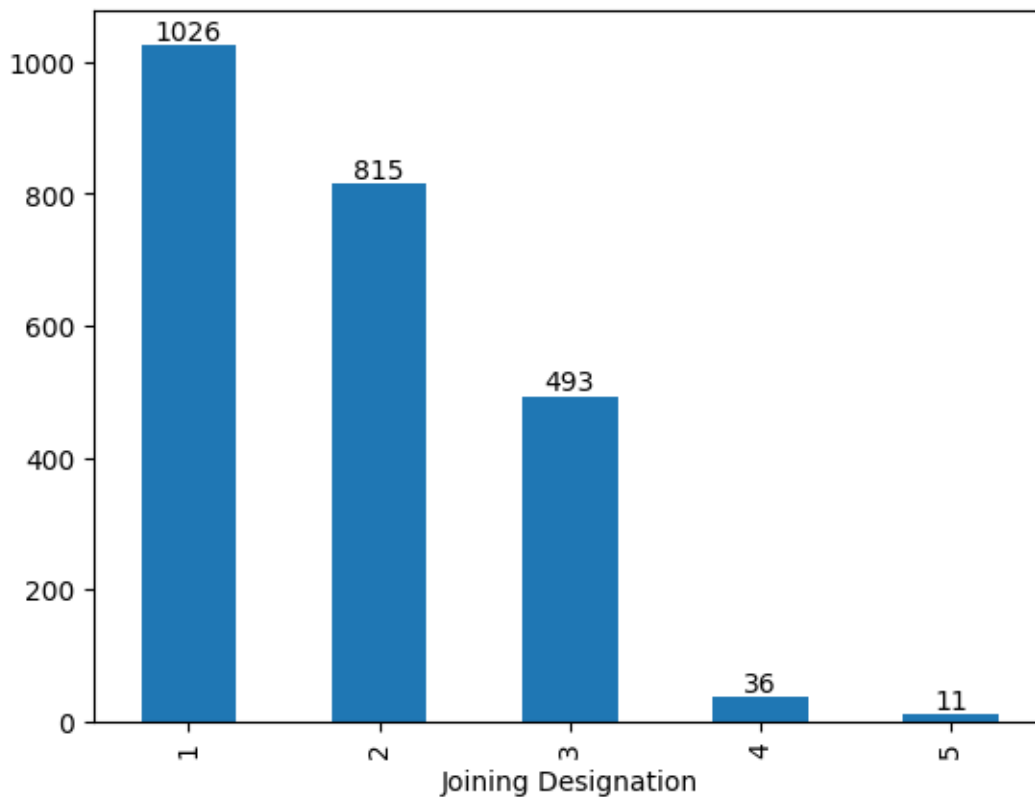


Insight

Maximum number of drivers joined in the year 2020

Maximum number of drivers joined in the month of July

```
[260]: ax = data.groupby('Driver_ID').agg({'Joining Designation': 'first'})['Joining_
↪Designation'].value_counts().plot(kind='bar')
ax.bar_label(ax.containers[0])
plt.show()
```

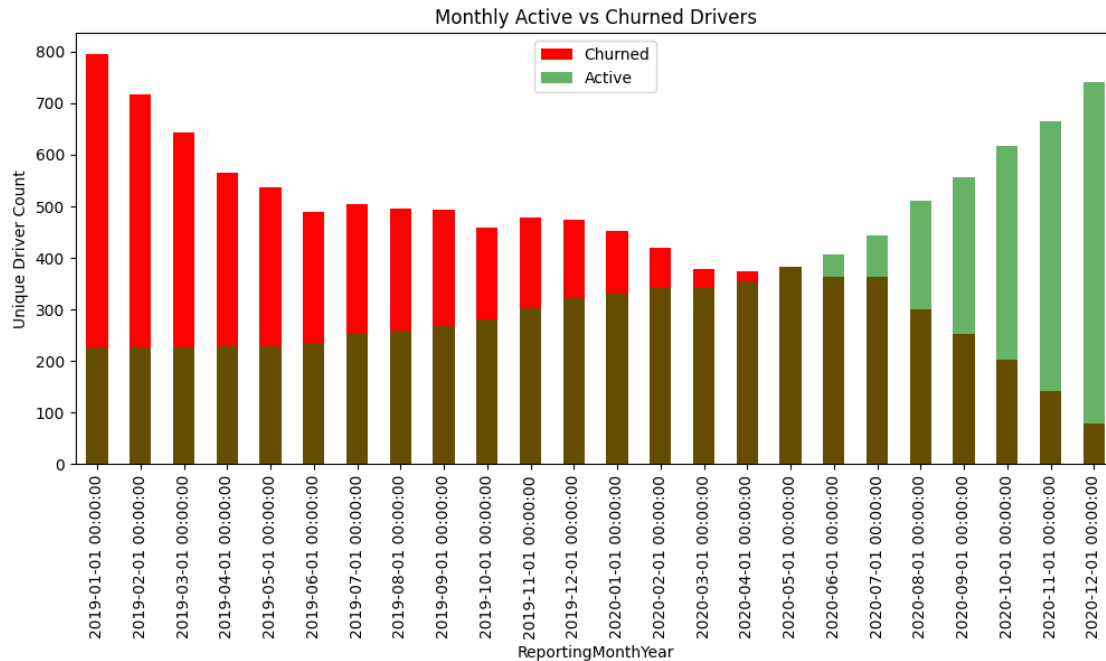


Insight

Maximum number of drivers, 1026, have a joining designation of 1

```
[261]: monthly_churn = data[data['Churn'] == 1].
        ↳groupby('ReportingMonthYear')['Driver_ID'].nunique()
monthly_active = data[data['Churn'] == 0].
        ↳groupby('ReportingMonthYear')['Driver_ID'].nunique()

plt.figure(figsize=(12,5))
monthly_churn.plot(kind='bar', color='red', label='Churned')
monthly_active.plot(kind='bar', color='green', alpha=0.6, label='Active')
plt.title('Monthly Active vs Churned Drivers')
plt.ylabel('Unique Driver Count')
plt.legend()
plt.show()
```

Insight

High early-year onboarding followed by significant early churn.

```
[262]: # Aggregate Grade First and Last
temp_df = (
    data.groupby('Driver_ID')['Grade']
        .agg(Grade_First='first', Grade_Last='last')
        .reset_index()
)

# Compute grade change (handle categorical)
temp_df['Grade_Change'] = (
    temp_df['Grade_Last'].cat.codes - temp_df['Grade_First'].cat.codes
)

# Plot First and Last Grade distribution
fig, axs = plt.subplots(1, 2, figsize=(10, 4))

ax1 = temp_df['Grade_First'].value_counts().sort_index().plot(
    kind='bar', ax=axs[0], title='Driver distribution based on the grade\nat_
    ↳first reporting date'
)
ax1.bar_label(ax1.containers[0])

ax2 = temp_df['Grade_Last'].value_counts().sort_index().plot(
```

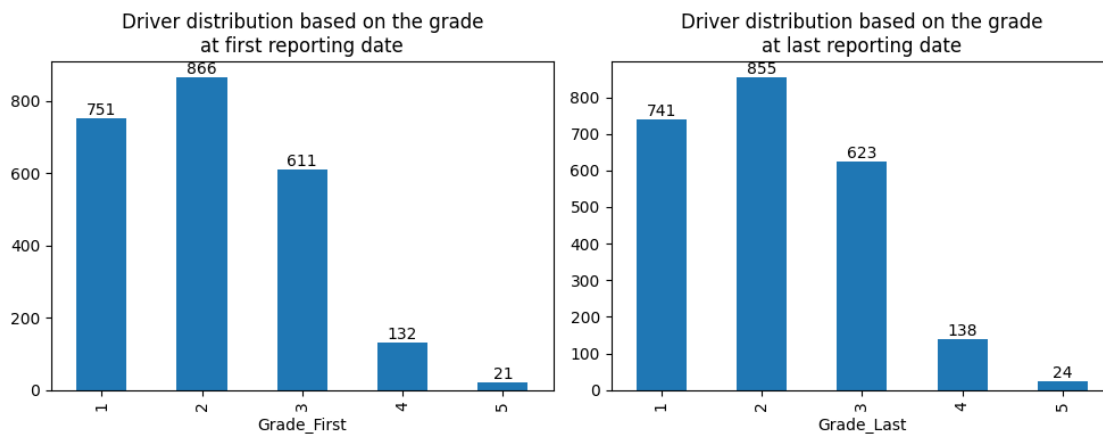
```

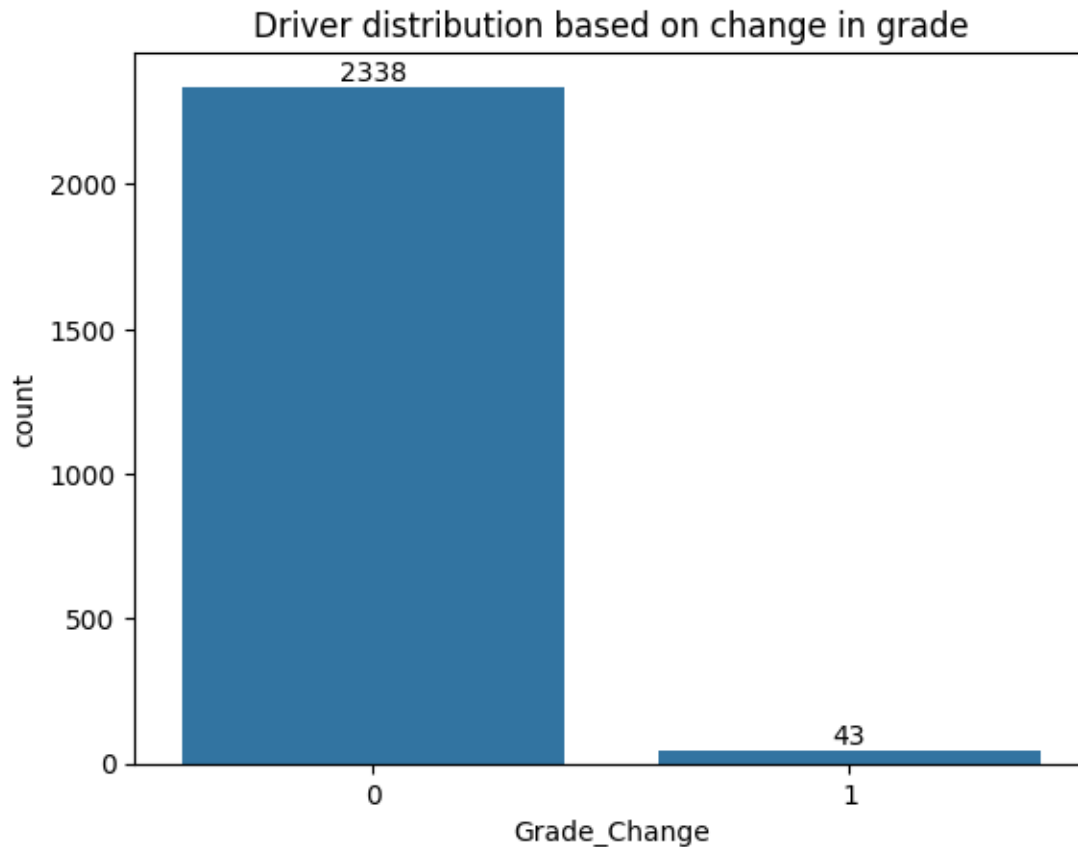
        kind='bar', ax=axes[1], title='Driver distribution based on the grade\nat_
        ↳last reporting date'
    )
    ax2.bar_label(ax2.containers[0])

plt.tight_layout()
plt.show()

# Plot Grade Change
ax = sns.countplot(data=temp_df, x='Grade_Change')
ax.set_title('Driver distribution based on change in grade')
ax.bar_label(ax.containers[0])
plt.show()

```

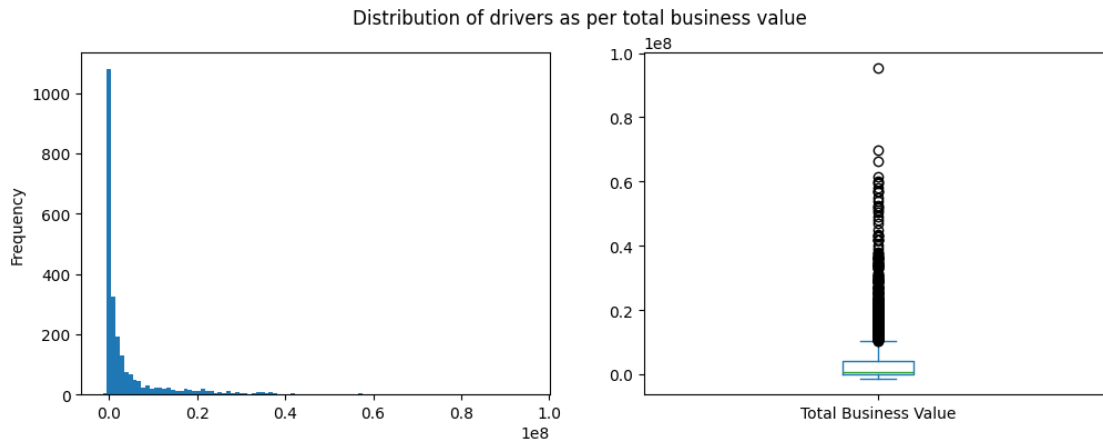




Insight

Maximum number of drivers have a grade of 2 and it doesn't change for the majority of the drivers

```
[263]: fig, axs = plt.subplots(1,2,figsize=(12,4))
temp_df = data.groupby('Driver_ID').agg({'Total Business Value':'sum'})['Total_
Business Value']
temp_df.plot(ax=axs[0], kind='hist', bins=100)
temp_df.plot(ax=axs[1], kind='box')
fig.suptitle('Distribution of drivers as per total business value')
plt.show()
```

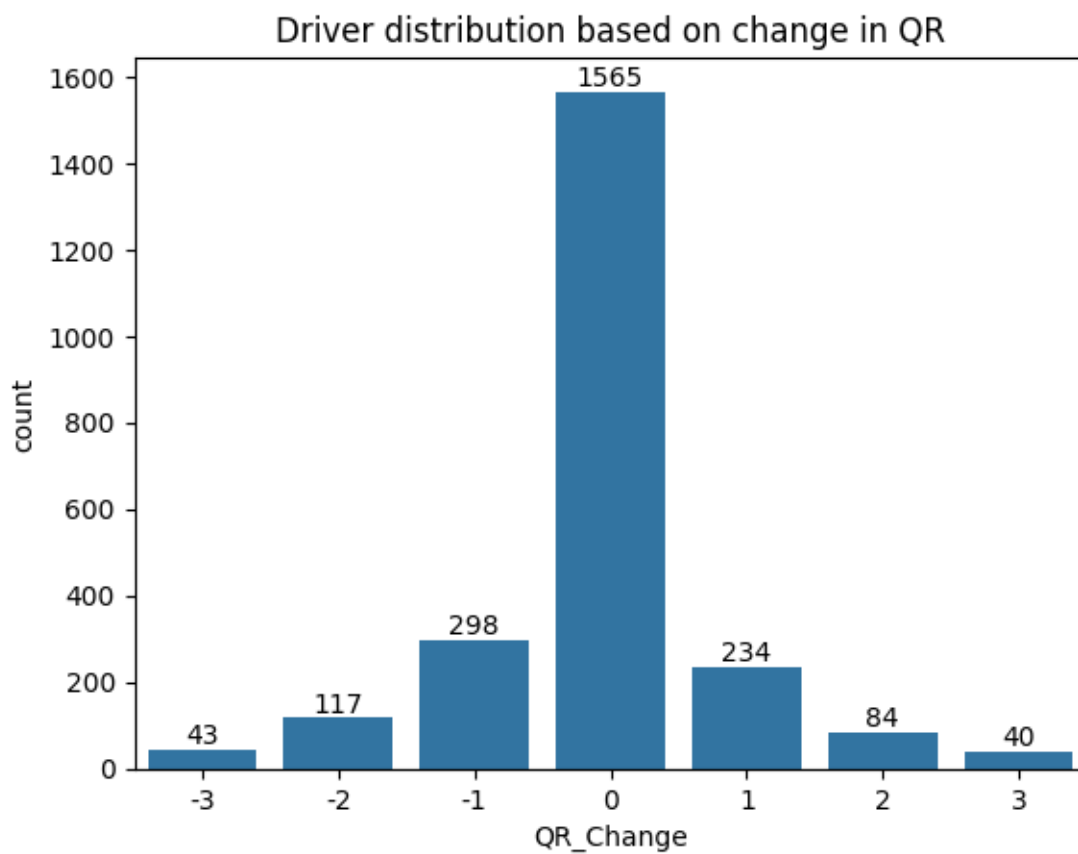
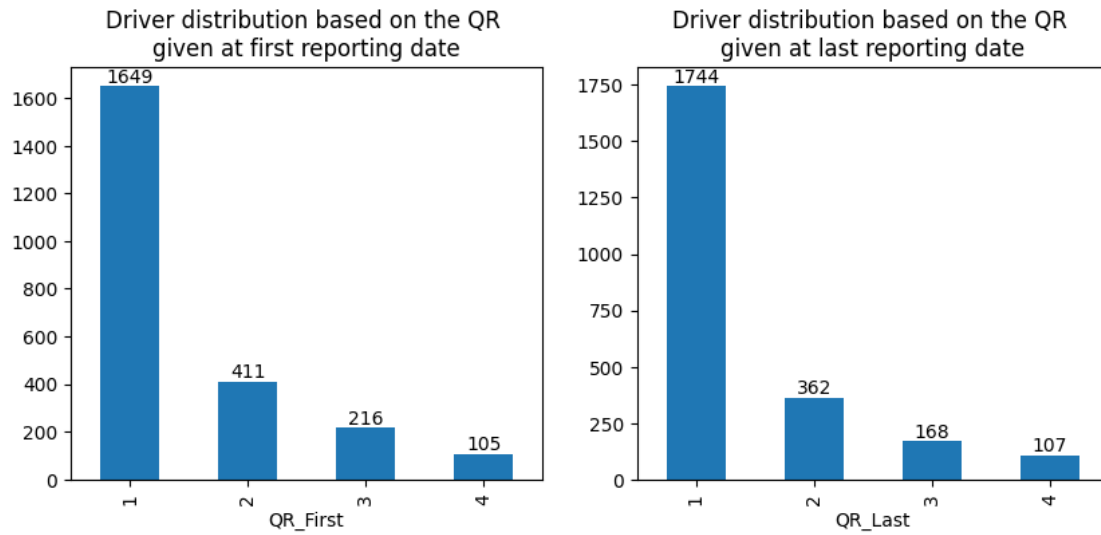


Insight

It is very evident that many drivers have a total business value of 0 and there are also a few drivers who have a -ve business value

The distribution is extremely right skewed

```
[264]: temp_df_1 = data.groupby('Driver_ID').agg({'Quarterly Rating':'first'}).
        ↪reset_index()
temp_df_1.rename(columns = {'Quarterly Rating':'QR_First'}, inplace=True)
temp_df_2 = data.groupby('Driver_ID').agg({'Quarterly Rating':'last'}).
        ↪reset_index()
temp_df_2.rename(columns = {'Quarterly Rating':'QR_Last'}, inplace=True)
temp_df = pd.merge(temp_df_1, temp_df_2, on='Driver_ID')
temp_df['QR_Change'] = temp_df['QR_Last'].astype('int') - temp_df['QR_First'].
        ↪astype('int')
fig, axs = plt.subplots(1,2,figsize=(10,4))
ax = temp_df['QR_First'].value_counts().plot(kind='bar', ax=axs[0],
        ↪title='Driver distribution based on the QR \ngiven at first reporting date')
ax.bar_label(ax.containers[0])
ax = temp_df['QR_Last'].value_counts().plot(kind='bar', ax=axs[1],
        ↪title='Driver distribution based on the QR \ngiven at last reporting date')
ax.bar_label(ax.containers[0])
plt.show()
ax = sns.countplot(data=temp_df, x = 'QR_Change')
ax.set_title('Driver distribution based on change in QR')
ax.bar_label(ax.containers[0])
plt.show()
```



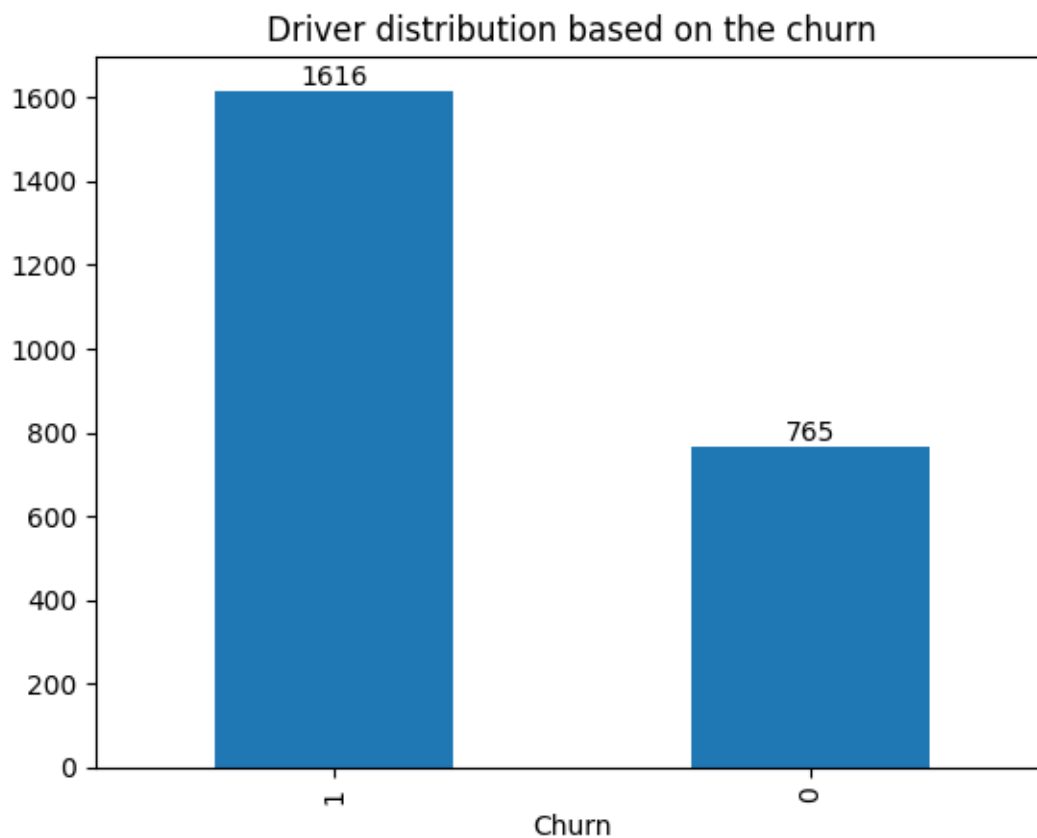
Insight

Majority of the drivers have a very low quarterly rating of 1

The change in QR plot shows that majority of the drivers don't see a change in their QR but there are decent number of drivers with positive change in QR and equally decent number of drivers with negative change in QR

There are no drivers with QR of 5

```
[265]: temp_df = data.groupby('Driver_ID').agg({'Churn': 'first'})['Churn']
ax = temp_df.value_counts().plot(kind='bar', title='Driver distribution based_
on the churn')
ax.bar_label(ax.containers[0])
plt.show()
(temp_df.value_counts(normalize=True)*100).round(0)
```



```
[265]: Churn
1      68.0
0      32.0
Name: proportion, dtype: float64
```

Insight) 1616 drivers have churned, which is around 68%

5 Bivariate analysis

```
[266]: driver_df = data.groupby('Driver_ID').agg({
    'ReportingMonthYear' : len,
    'Age' : 'last',
    'Gender' : 'first',
    'City' : 'first',
    'Education_Level' : 'first',
    'Income' : 'last',
    'Dateofjoining' : 'first',
    'LastWorkingDate' : 'last',
    'Joining Designation' : 'first',
    'Grade' : 'last',
    'Total Business Value' : 'sum',
    'Quarterly Rating' : 'last',
    'Churn': 'last'
}).reset_index()
driver_df.rename(columns={'ReportingMonthYear': 'Months of Service'},
    inplace=True)
driver_df.head(10)
```

```
[266]:
```

	Driver_ID	Months of Service	Age	Gender	City	Education_Level	Income \
0	1	3	28.0	Male	C23	Graduate	57387
1	2	2	31.0	Male	C7	Graduate	67016
2	4	5	43.0	Male	C13	Graduate	65603
3	5	3	29.0	Male	C9	10+	46368
4	6	5	31.0	Female	C11	12+	78728
5	8	3	34.0	Male	C2	10+	70656
6	11	1	28.0	Female	C19	Graduate	42172
7	12	6	35.0	Male	C23	Graduate	28116
8	13	23	31.0	Male	C19	Graduate	119227
9	14	3	39.0	Female	C26	10+	19734

	Dateofjoining	LastWorkingDate	Joining Designation	Grade \
0	2018-12-24	2019-11-03	1	1
1	2020-06-11	NaT	2	2
2	2019-07-12	2020-04-27	2	2
3	2019-09-01	2019-07-03	1	1
4	2020-07-31	NaT	3	3
5	2020-09-19	2020-11-15	3	3
6	2020-07-12	NaT	1	1
7	2019-06-29	2019-12-21	1	1
8	2015-05-28	2020-11-25	1	4
9	2020-10-16	NaT	3	3

	Total Business Value	Quarterly Rating	Churn
0	1715580	2	1

1	0	1	0
2	350000	1	1
3	120360	1	1
4	1265000	2	0
5	0	1	1
6	0	1	0
7	2607180	1	1
8	10213040	1	1
9	0	1	0

```
[267]: drivers_with_2_year_service = driver_df[driver_df['Months of Service'] == 24]
↳ ['Driver_ID'].reset_index(drop=True)
```

```
[268]: def calculate_change(df, column):
    """Calculates first, last, and change of a column for each Driver_ID."""
    grouped = df.groupby('Driver_ID')[column]
    change_df = grouped.first().to_frame(name=f'{column}_First')
    change_df[f'{column}_Last'] = grouped.last()

    # Ensure data is numeric before subtraction
    change_df[f'{column}_First'] = pd.to_numeric(change_df[f'{column}_First'],
↳ errors='coerce')
    change_df[f'{column}_Last'] = pd.to_numeric(change_df[f'{column}_Last'],
↳ errors='coerce')

    change_df[f'{column}_Change'] = change_df[f'{column}_Last'] -
↳ change_df[f'{column}_First']
    return change_df[[f'{column}_Change']].reset_index()

# Columns for which to calculate change
change_columns = ['Income', 'Grade', 'Quarterly Rating']

# Compute all in a loop and merge
for col in change_columns:
    temp_df = calculate_change(df, col)
    driver_df = driver_df.merge(temp_df, on='Driver_ID', how='left')

# Create additional binary column for improvement
driver_df['Quarterly Rating Improved'] = (driver_df['Quarterly Rating_Change']
↳ 0).astype(int)

driver_df.head()
```

```
[268]: Driver_ID  Months of Service  Age  Gender  City  Education_Level  Income \
0          1          3  28.0   Male  C23      Graduate    57387
1          2          2  31.0   Male   C7      Graduate    67016
2          4          5  43.0   Male  C13      Graduate    65603
3          5          3  29.0   Male   C9      10+      46368
```


4	6	5	31.0	Female	C11	12+	78728
---	---	---	------	--------	-----	-----	-------

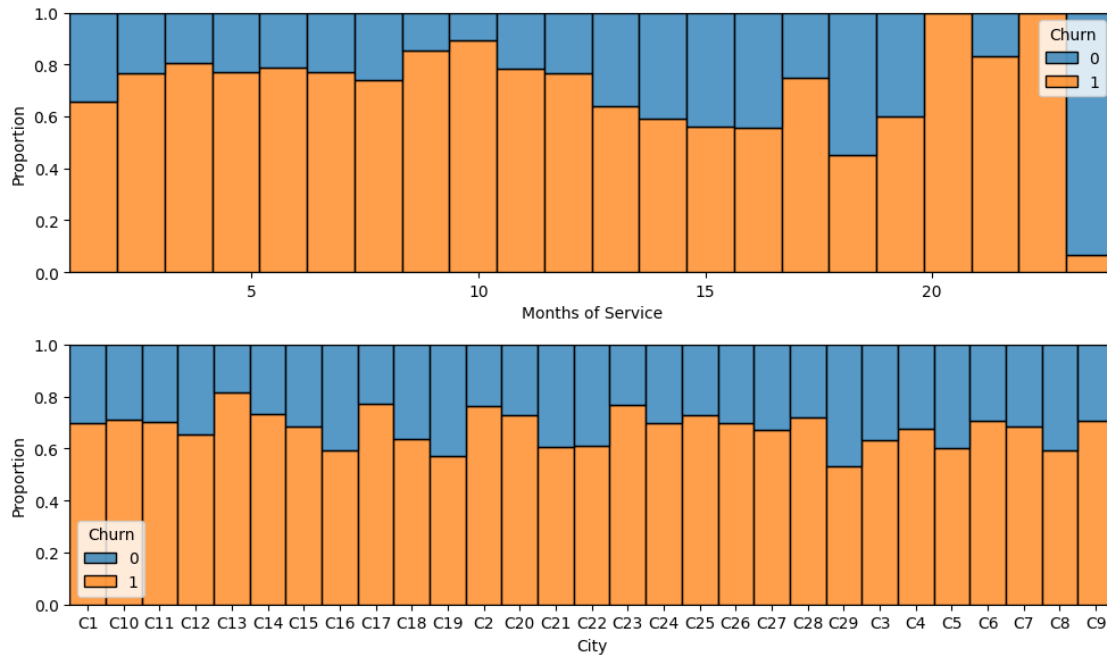
	Dateofjoining	LastWorkingDate	Joining	Designation	Grade	\
0	2018-12-24	2019-11-03			1	1
1	2020-06-11	NaT			2	2
2	2019-07-12	2020-04-27			2	2
3	2019-09-01	2019-07-03			1	1
4	2020-07-31	NaT			3	3

	Total Business Value	Quarterly Rating	Churn	Income_Change	Grade_Change	\
0	1715580	2	1	0	0	
1	0	1	0	0	0	
2	350000	1	1	0	0	
3	120360	1	1	0	0	
4	1265000	2	0	0	0	

	Quarterly Rating_Change	Quarterly Rating Improved
0	0	0
1	0	0
2	0	0
3	0	0
4	1	1

```
[269]: driver_df['Income_Raise'] = driver_df['Income_Change'].apply(lambda x: 1 if x>0
↪else 0)
```

```
[270]: fig, axs = plt.subplots(2,1,figsize=(10,6))
sns.histplot(ax = axs[0], data=driver_df, x='Months of Service', hue='Churn',
↪stat="proportion", multiple="fill")
sns.histplot(ax = axs[1], data=driver_df, x='City', hue='Churn',
↪stat="proportion", multiple="fill")
plt.tight_layout()
plt.show()
```

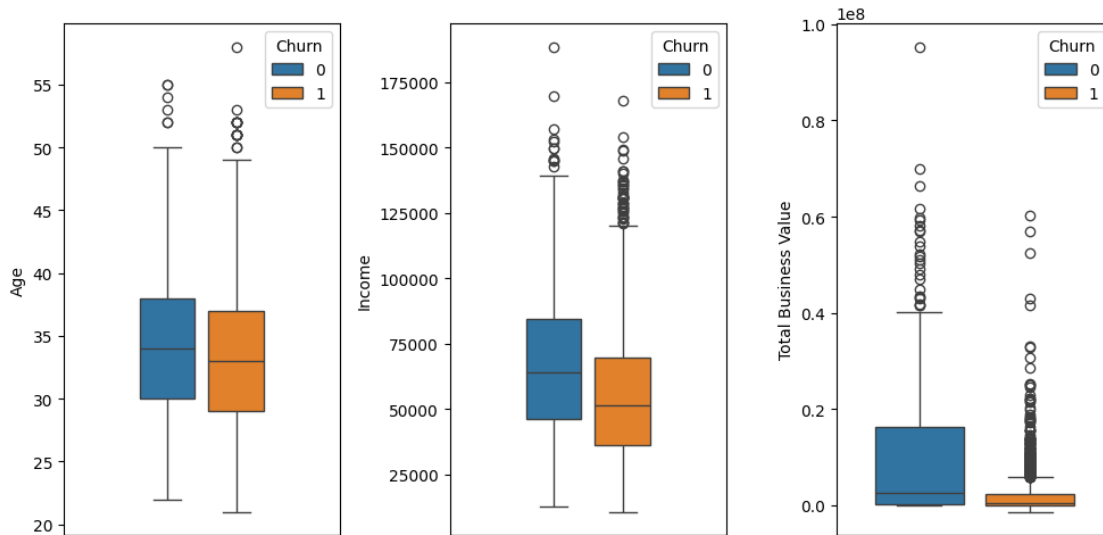


Insight

The churn rate is generally higher in drivers with less months of service and low in drivers with longer months of service with exception for 21, 22 and 23 months of service where the churn rates seems to be very high

The city C13 has the highest churn rate and city C29 has the lowest churn rate

```
[271]: fig, axs = plt.subplots(1,3,figsize=(10,5))
sns.boxplot(ax=axs[0], data=driver_df, y='Age', hue='Churn', width=0.5, gap=0.2)
sns.boxplot(ax=axs[1], data=driver_df, y='Income', hue='Churn', width=0.5,
    ↪gap=0.2)
sns.boxplot(ax=axs[2], data=driver_df, y='Total Business Value', hue='Churn',
    ↪gap=0.2)
plt.tight_layout()
plt.show()
```



Insight

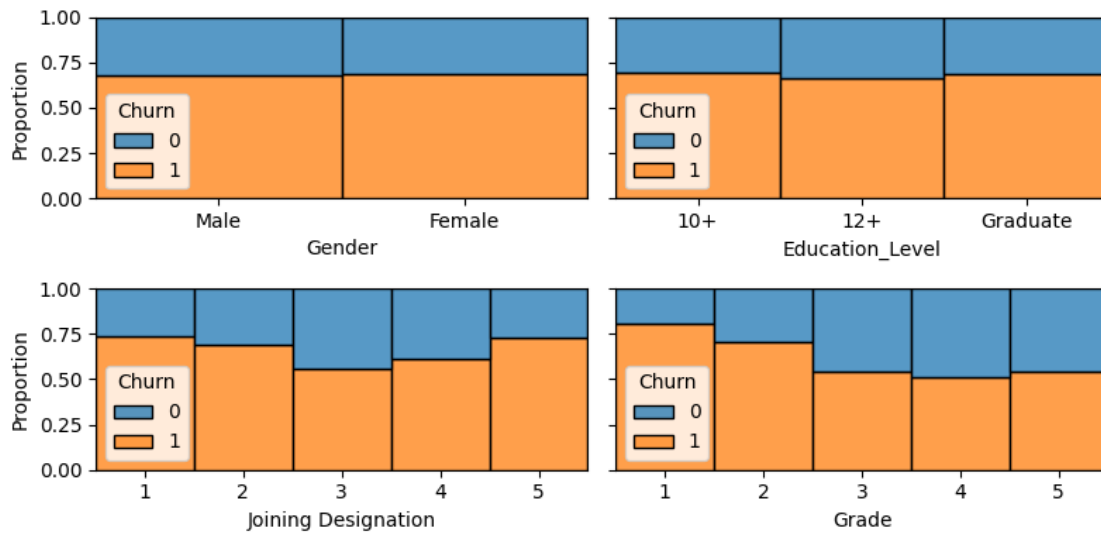
The median age of drivers who have churned is slightly lesser than that of the drivers who have not churned

The median income of drivers who have churned is lesser than that of the drivers who have not churned

The median Total Business Value of drivers who have churned is lesser than that of the drivers who have not churned

The drivers who have churned also had -ve Total Business Value

```
[272]: fig, axs = plt.subplots(2,2,figsize=(8,4),sharey=True)
sns.histplot(ax=axs[0,0], data=driver_df, x='Gender', hue='Churn',
             stat="proportion", multiple="fill")
sns.histplot(ax=axs[0,1], data=driver_df, x='Education_Level', hue='Churn',
             stat='proportion', multiple='fill')
sns.histplot(ax=axs[1,0], data=driver_df, x='Joining Designation', hue='Churn',
             stat='proportion', multiple='fill')
sns.histplot(ax=axs[1,1], data=driver_df, x='Grade', hue='Churn',
             stat='proportion', multiple='fill')
plt.tight_layout()
plt.show()
```



Insight

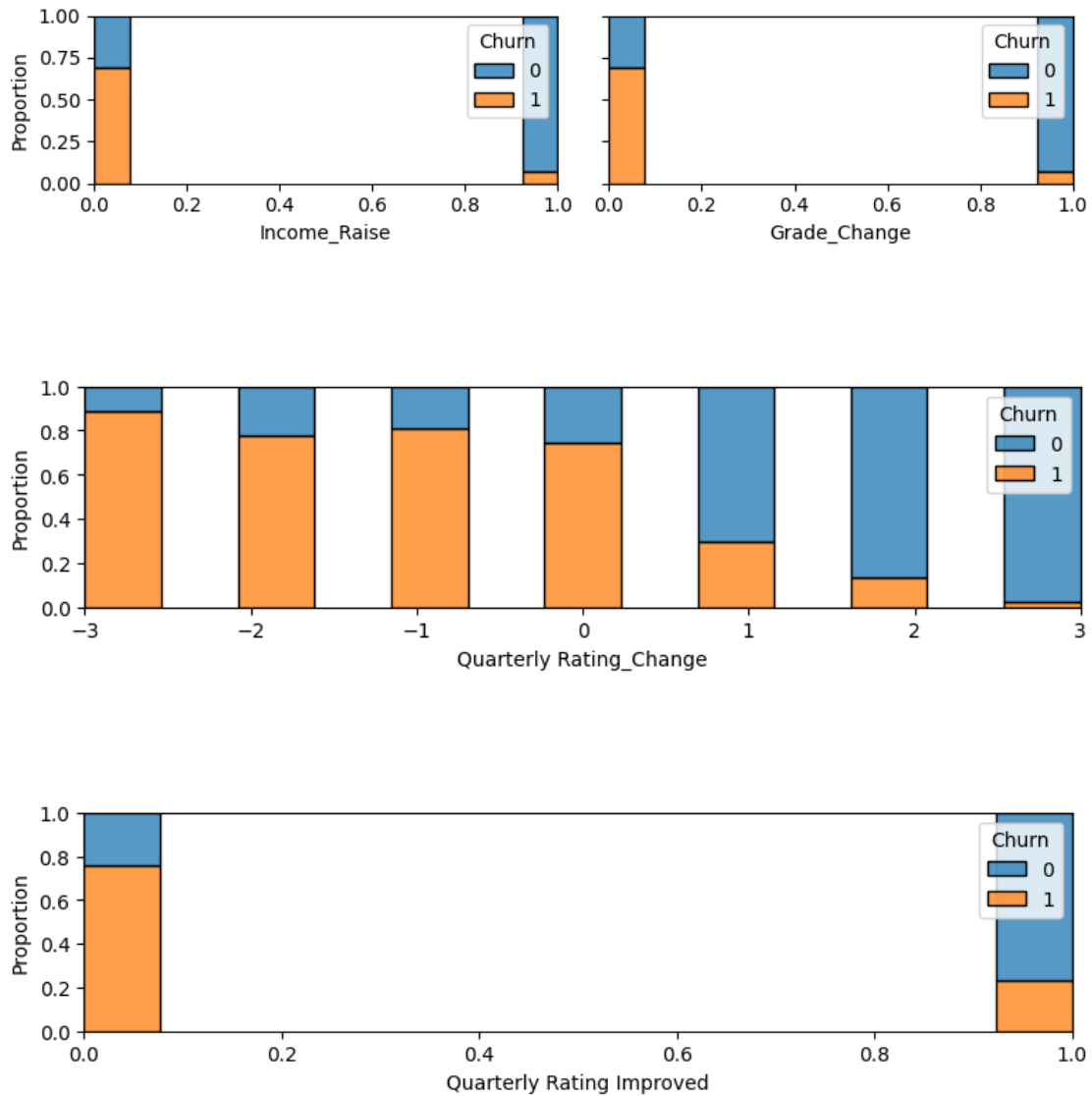
The churn rate is almost equal in both male and female drivers

The churn rate is almost equal in 10+ and Graduates and slightly lower in 12+

The churn rate is less for joining designation 3

The churn rate is less for higher grades

```
[273]: fig, axs = plt.subplots(1,2,figsize=(8,2),sharey=True)
sns.histplot(ax=axs[0], data=driver_df, x='Income_Raise', hue='Churn',
             stat='proportion', multiple='fill')
sns.histplot(ax=axs[1], data=driver_df, x='Grade_Change', hue='Churn',
             stat='proportion', multiple='fill')
plt.tight_layout()
plt.show()
plt.figure(figsize=(9,2))
sns.histplot(data=driver_df, x='Quarterly Rating_Change', hue='Churn',
             stat='proportion', multiple='fill')
plt.show()
plt.figure(figsize=(9,2))
sns.histplot(data=driver_df, x='Quarterly Rating_Improved', hue='Churn',
             stat='proportion', multiple='fill')
plt.show()
```



Insight

The churn rate is very less in drivers whose income has raised

The churn rate is very less in drivers whose grade has raised

The churn rate is very less in drivers whose Quarterly rating has increased

[275]: `data.columns`

[275]: `Index(['ReportingMonthYear', 'Driver_ID', 'Age', 'Gender', 'City',
'Education_Level', 'Income', 'Dateofjoining', 'LastWorkingDate',
'Joining Designation', 'Grade', 'Total Business Value',
'Quarterly Rating', 'Monthofjoining', 'Yearofjoining', 'Churn'],`

```
dtype='object')
```

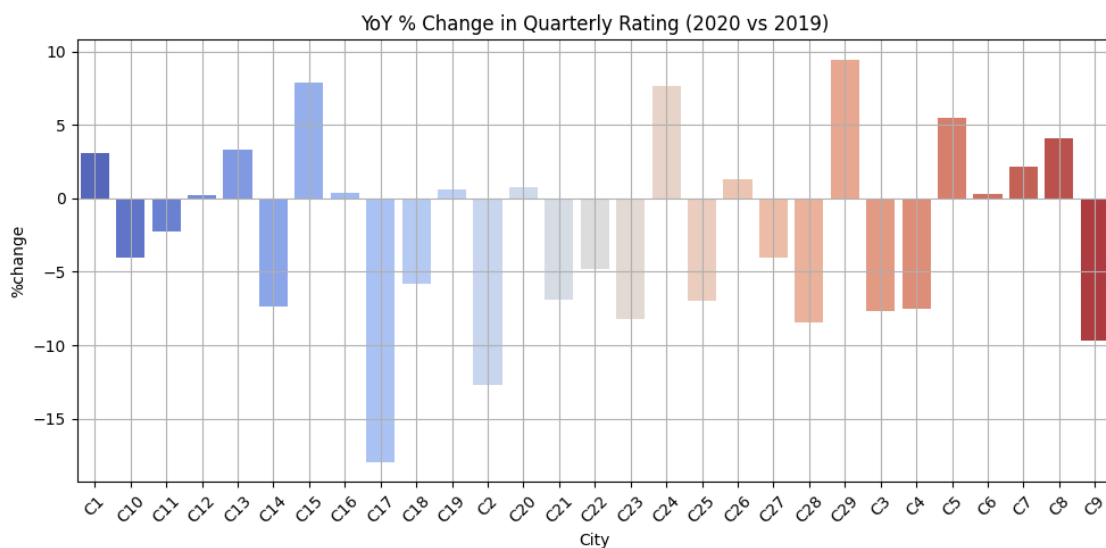
```
[278]: temp_df = data.copy()
temp_df['ReportingYear'] = temp_df['ReportingMonthYear'].dt.year

# Now group by year instead of full date
temp_df = temp_df.groupby(['City', 'ReportingYear'])['Quarterly Rating'].mean().
    ↪reset_index()

# Pivot: one row per city, one column per year
pivot_df = temp_df.pivot(index='City', columns='ReportingYear',
    ↪values='Quarterly Rating').reset_index()

# Optional: rename columns if needed (not strictly necessary now)
pivot_df.columns.name = None # remove pivot's column name
if 2019 in pivot_df.columns and 2020 in pivot_df.columns:
    pivot_df['%change'] = ((pivot_df[2020] - pivot_df[2019]) / pivot_df[2019] *
    ↪100).round(2)
else:
    pivot_df['%change'] = None # or raise an error/warning

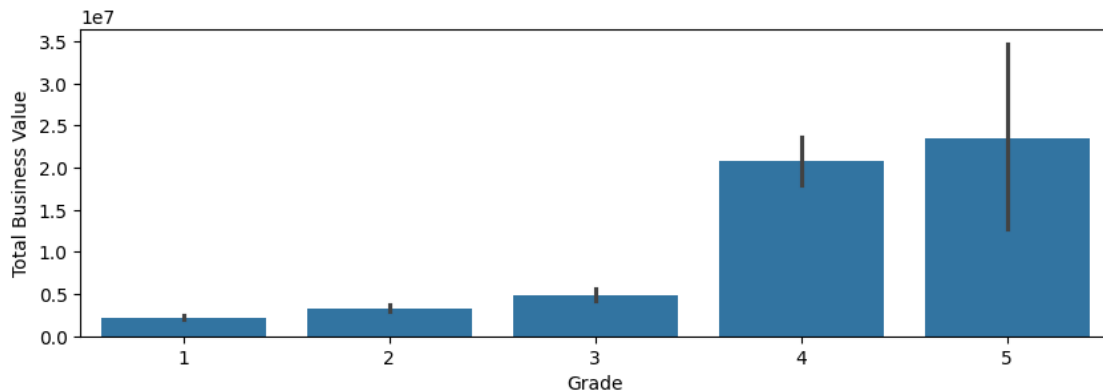
# Plot
plt.figure(figsize=(10,5))
sns.barplot(data=pivot_df, x='City', y='%change', palette='coolwarm')
plt.title("YoY % Change in Quarterly Rating (2020 vs 2019)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.show()
```



Insight

The city C29 shows most improvement in Quarterly Rating in 2020 compared to 2019

```
[279]: plt.figure(figsize=(10,3))
sns.barplot(data=driver_df, x='Grade', y='Total Business Value',
            estimator='mean')
plt.show()
print('Mean of Total Business Value of drivers with grade 5:',
      driver_df[driver_df['Grade'] == 5]['Total Business Value'].sum())
```



Mean of Total Business Value of drivers with grade 5: 565760460

Insight

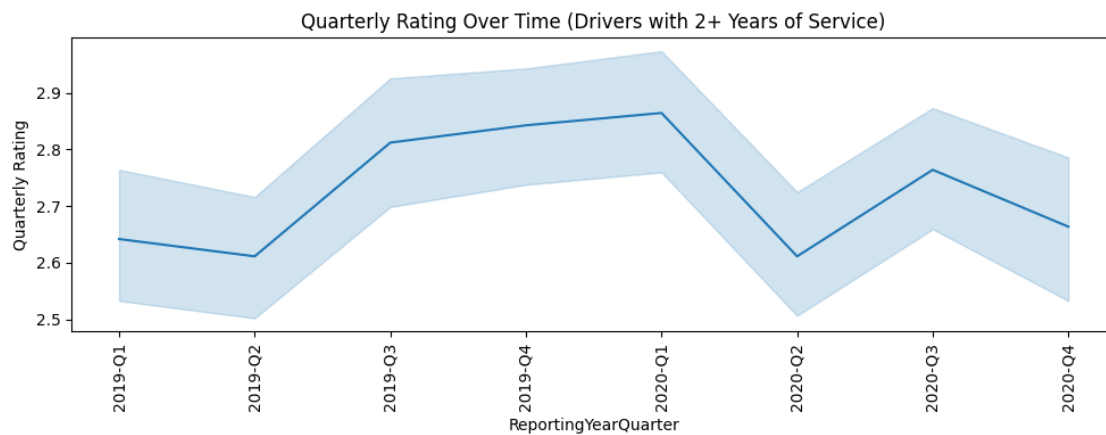
The mean of Total Business Value of drivers with grade 5 is higher than those with other grades

```
[280]: # Efficiently create 'ReportingYearQuarter'
data['ReportingYearQuarter'] = data['ReportingMonthYear'].dt.to_period('Q').
      astype(str).str.replace('Q', '-Q')

# Filter for drivers with 2-year service
temp_driver_full_service_df = data[data['Driver_ID'].
      isin(drivers_with_2_year_service)]

# Group and aggregate
agg_df = temp_driver_full_service_df.groupby(
      ['Driver_ID', 'ReportingYearQuarter']
).agg({
      'Quarterly Rating': 'last',
      'Total Business Value': 'sum'
}).reset_index()
```

```
# Plot
plt.figure(figsize=(10,4))
sns.lineplot(data=agg_df, x='ReportingYearQuarter', y='Quarterly Rating')
plt.xticks(rotation=90)
plt.title("Quarterly Rating Over Time (Drivers with 2+ Years of Service)")
plt.tight_layout()
plt.show()
```



Insight

There is a dip in the quarterly rating in Q2 and then it increases in Q3.

This pattern can be observed for both the years

```
[281]: # Filter relevant data
temp_driver_full_service_df = data[data['Driver_ID'].
    ↪isin(drivers_with_2_year_service)]

# Select top N drivers (first N unique IDs)
sample_driver_ids = temp_driver_full_service_df['Driver_ID'].unique()[:20]

# Plot loop
for driver_id in sample_driver_ids:
    sample_df = ↪
    ↪temp_driver_full_service_df[temp_driver_full_service_df['Driver_ID'] == ↪
    ↪driver_id]

    fig, axs = plt.subplots(2, 1, figsize=(10, 5), sharex=True)

    # Quarterly Rating
    sns.barplot(ax=axs[0], data=sample_df, x='ReportingMonthYear', y='Quarterly_
    ↪Rating', color='skyblue')
    axs[0].set_title('Quarterly Rating')
```



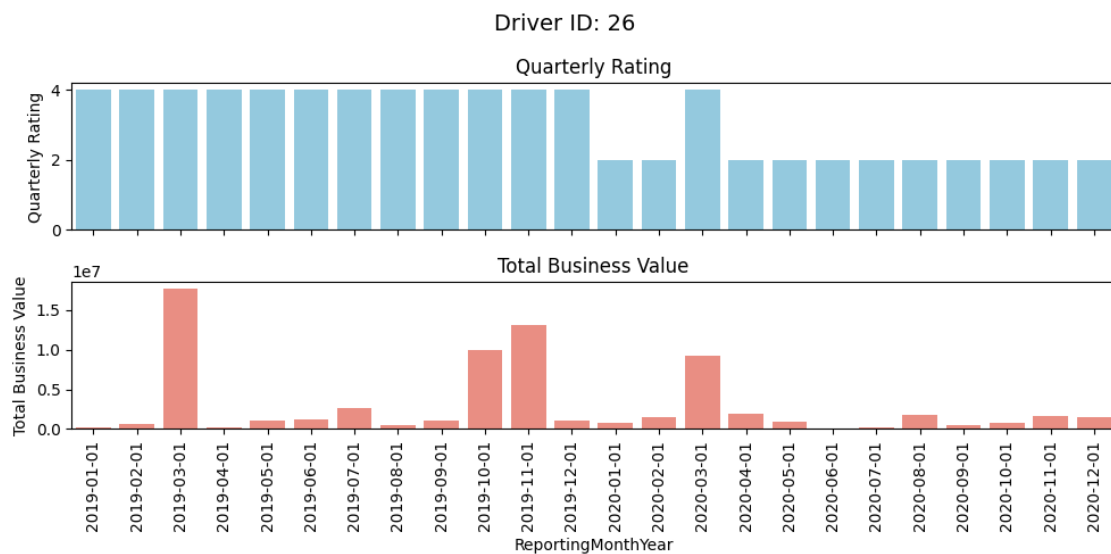
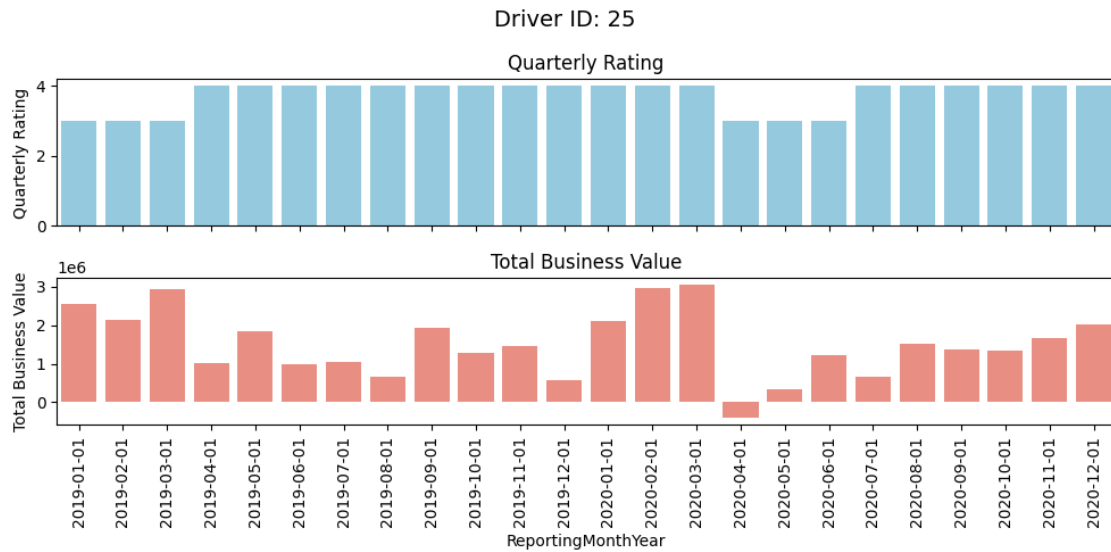
```

axs[0].tick_params(axis='x', rotation=90)

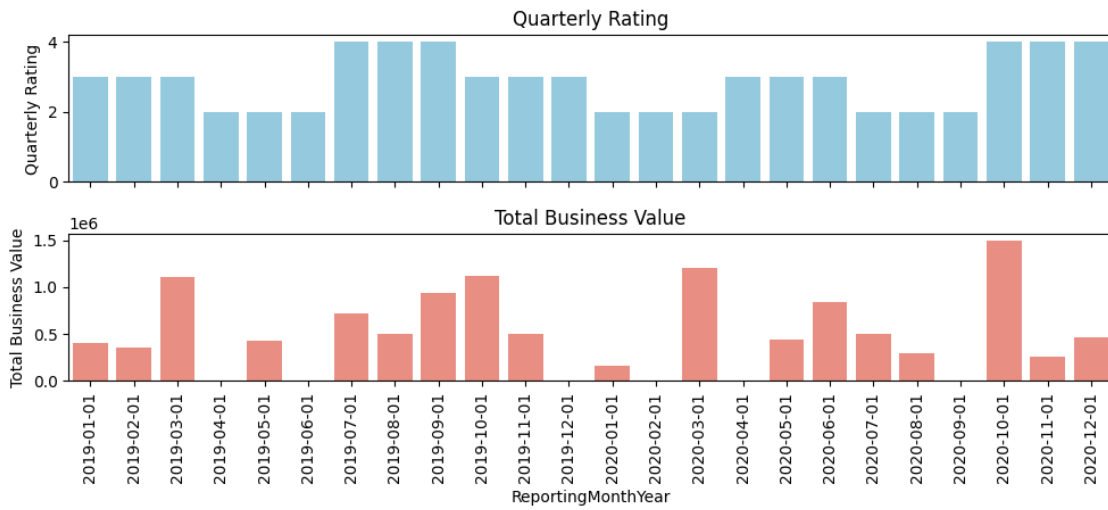
# Total Business Value
sns.barplot(ax=axs[1], data=sample_df, x='ReportingMonthYear', y='Total_
Business Value', color='salmon')
axs[1].set_title('Total Business Value')
axs[1].tick_params(axis='x', rotation=90)

fig.suptitle(f'Driver ID: {driver_id}', fontsize=14)
plt.tight_layout()
plt.show()

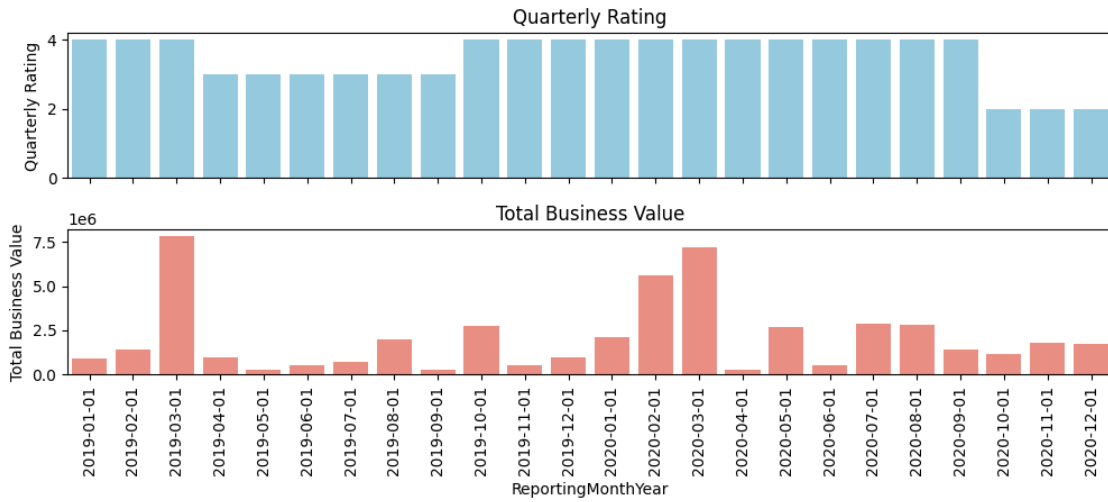
```



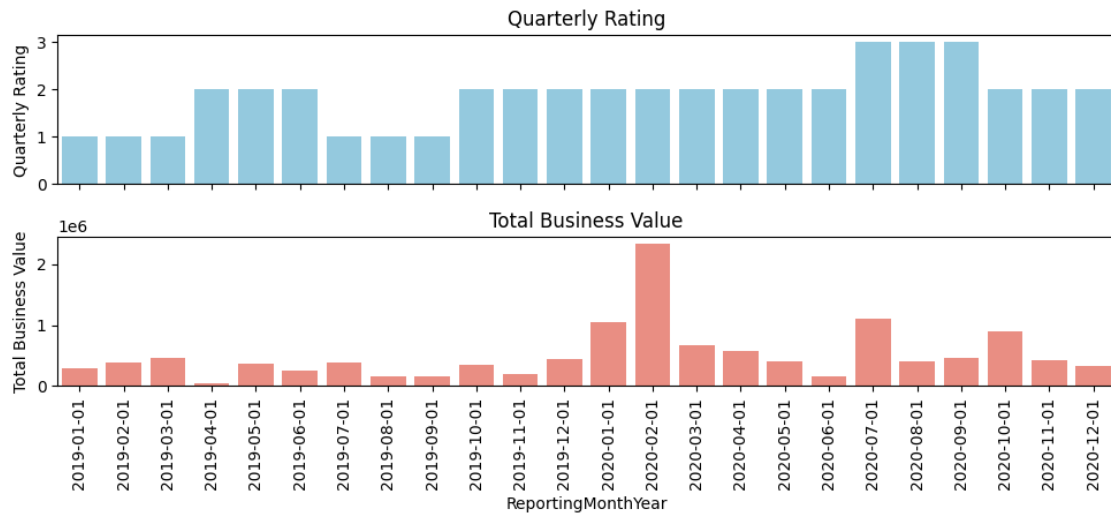
Driver ID: 56



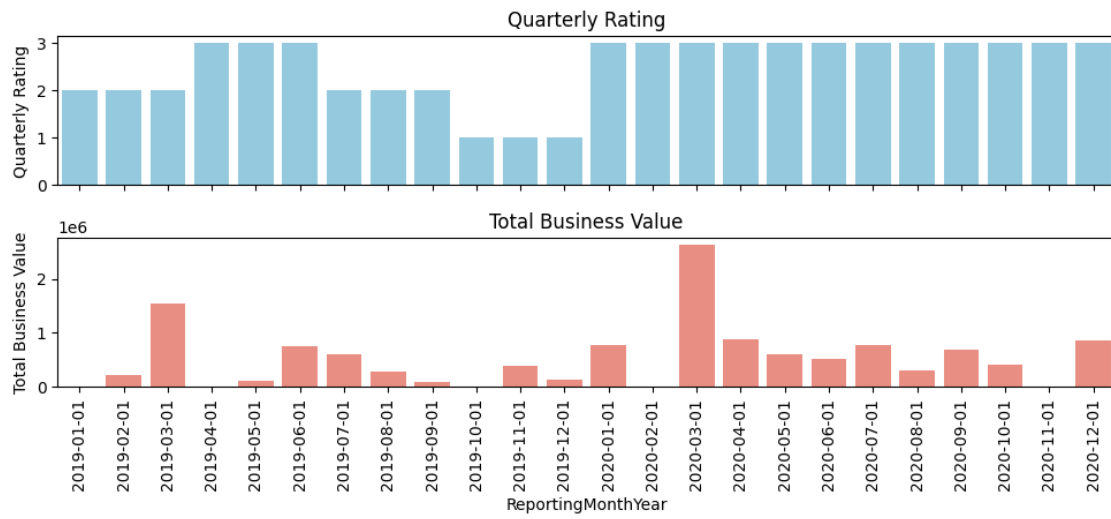
Driver ID: 60



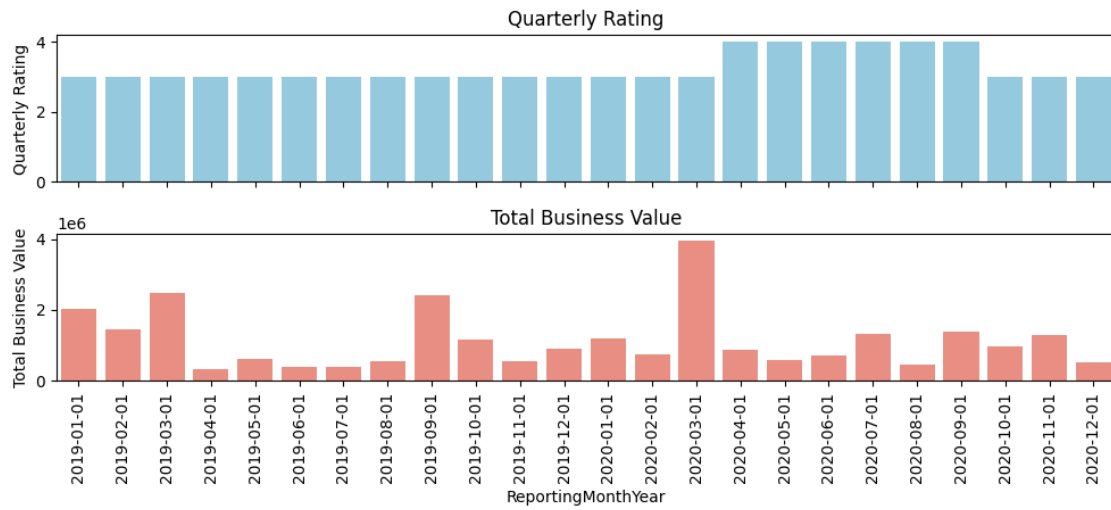
Driver ID: 63



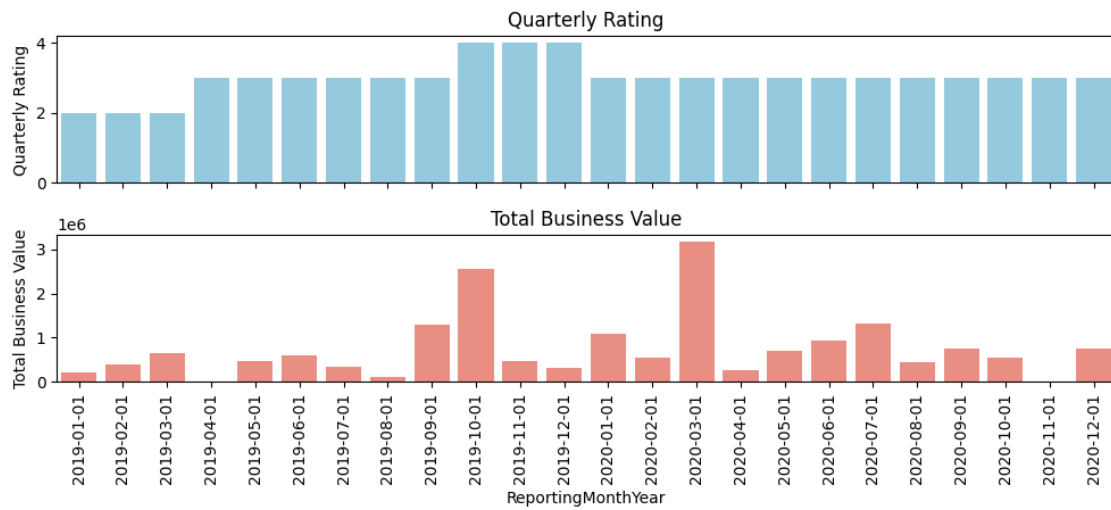
Driver ID: 67



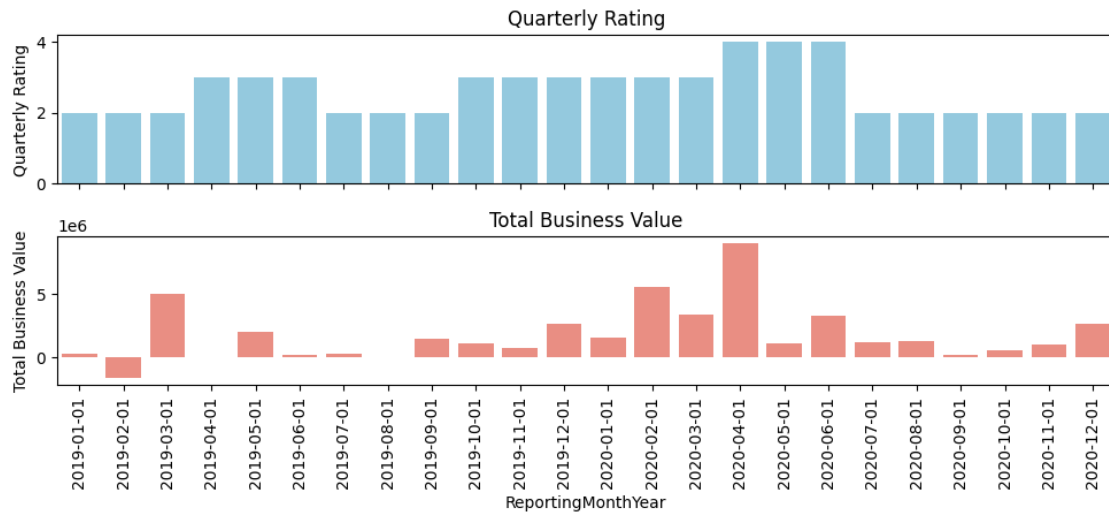
Driver ID: 68



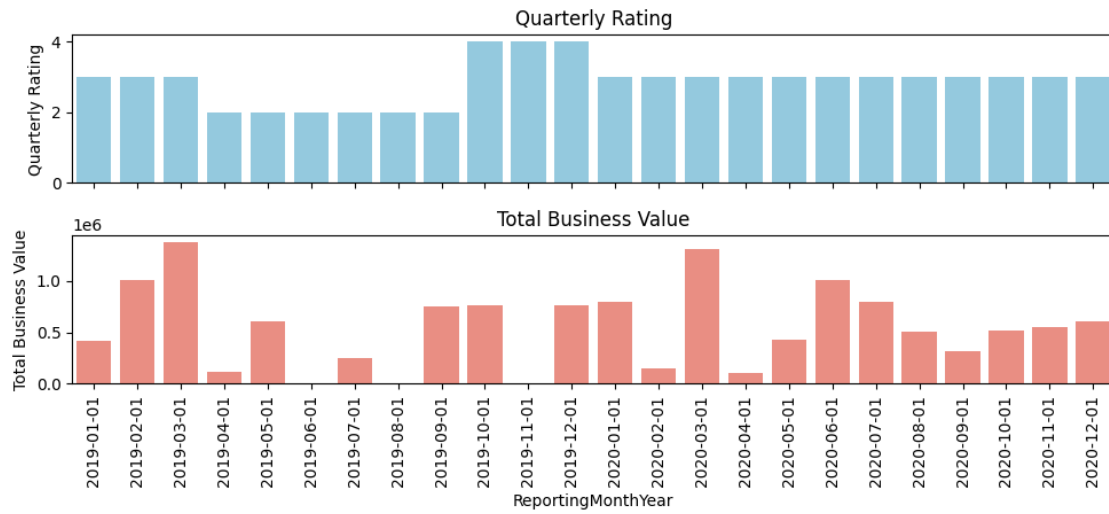
Driver ID: 77



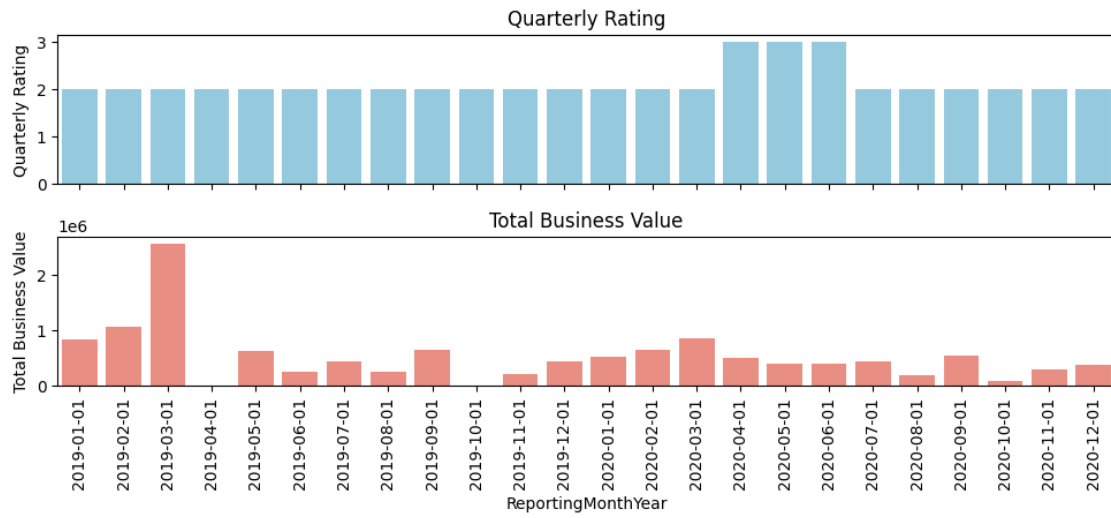
Driver ID: 78



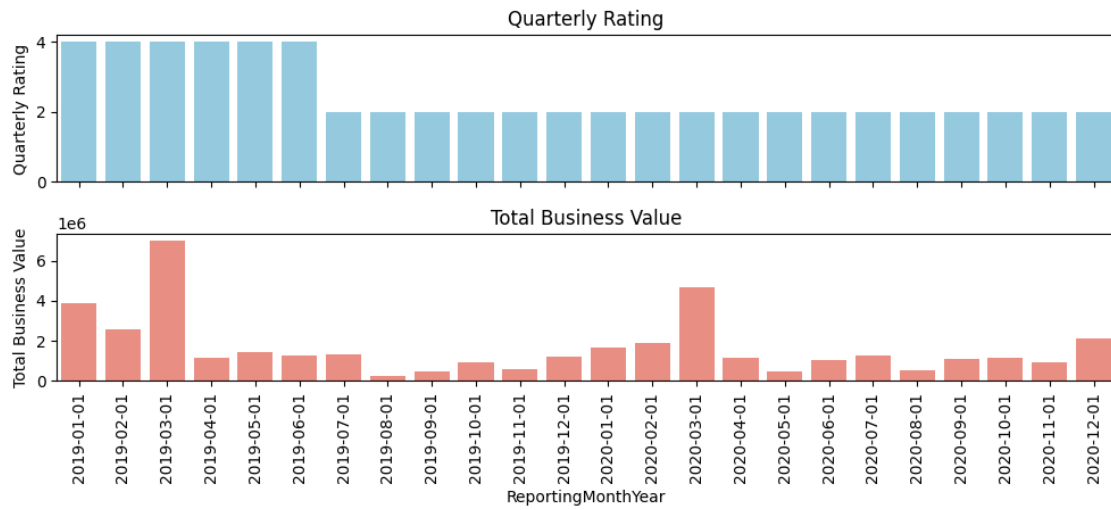
Driver ID: 112



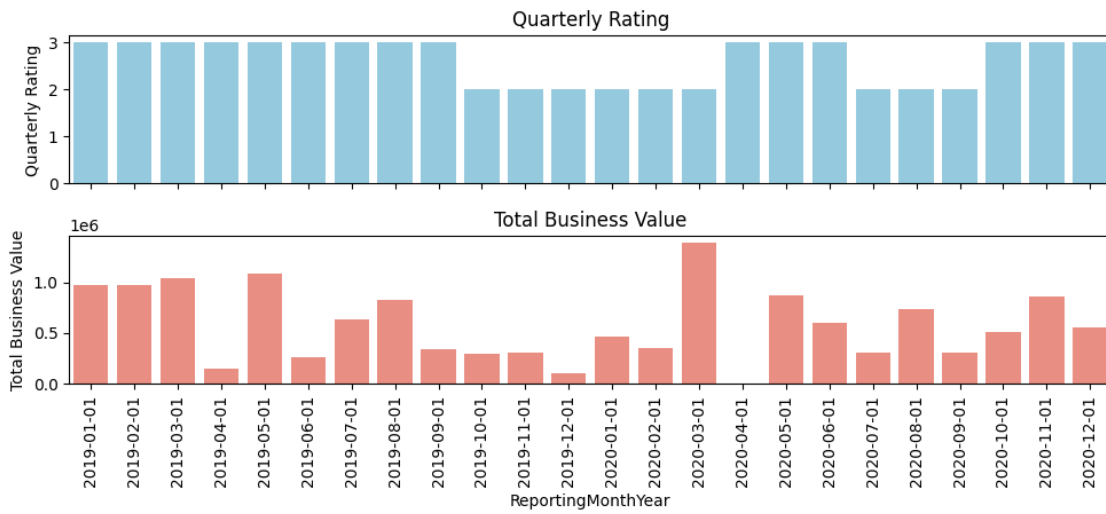
Driver ID: 115



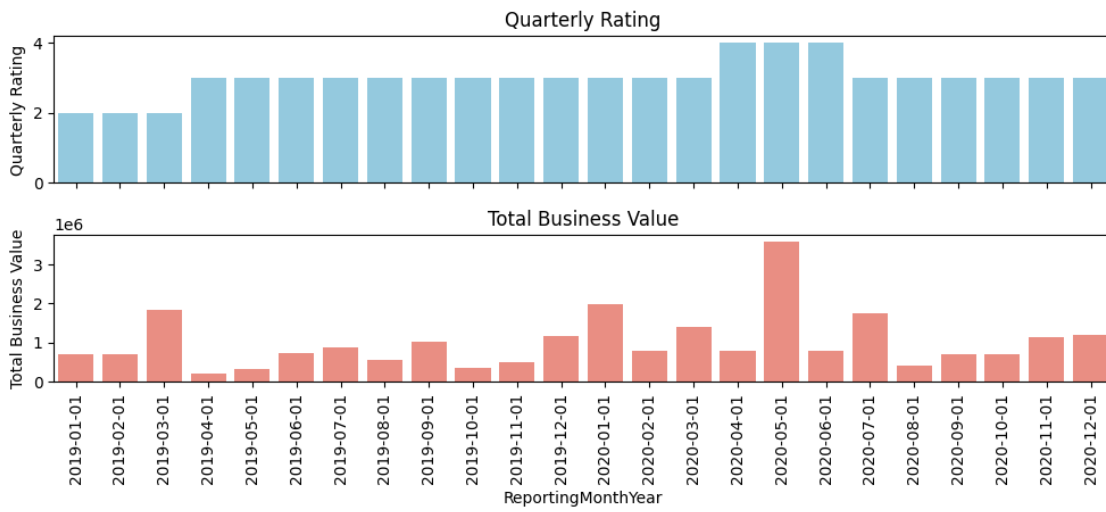
Driver ID: 117



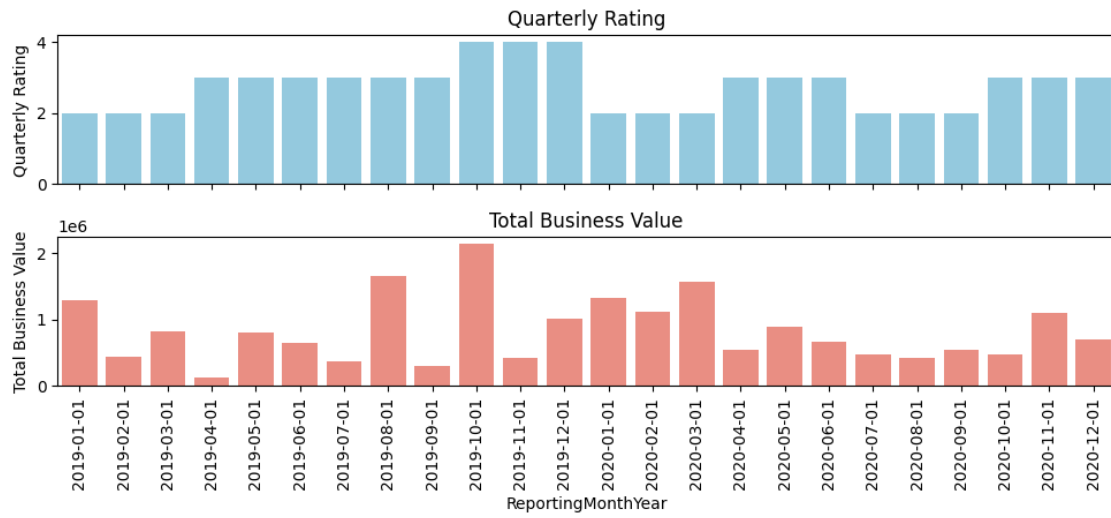
Driver ID: 140



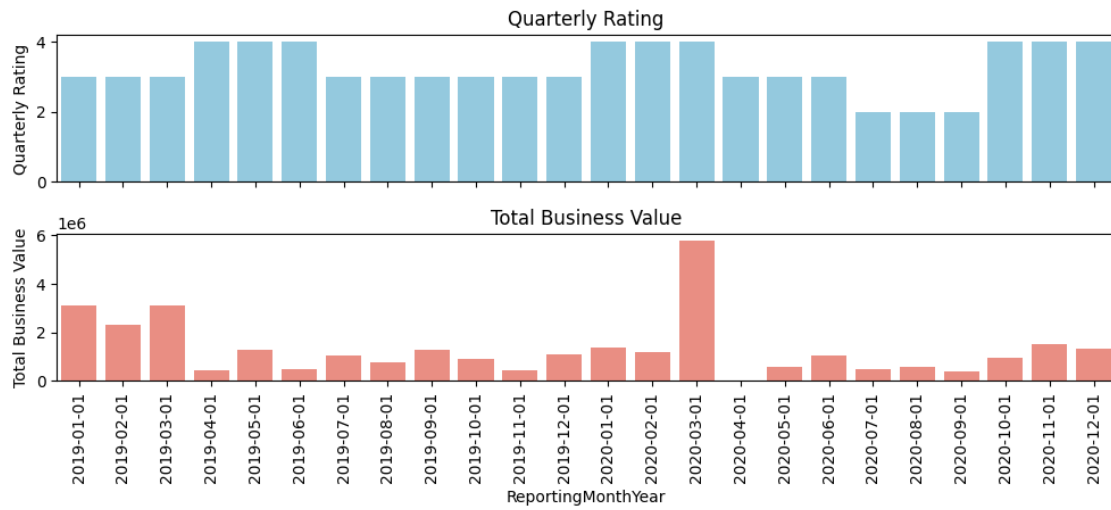
Driver ID: 150



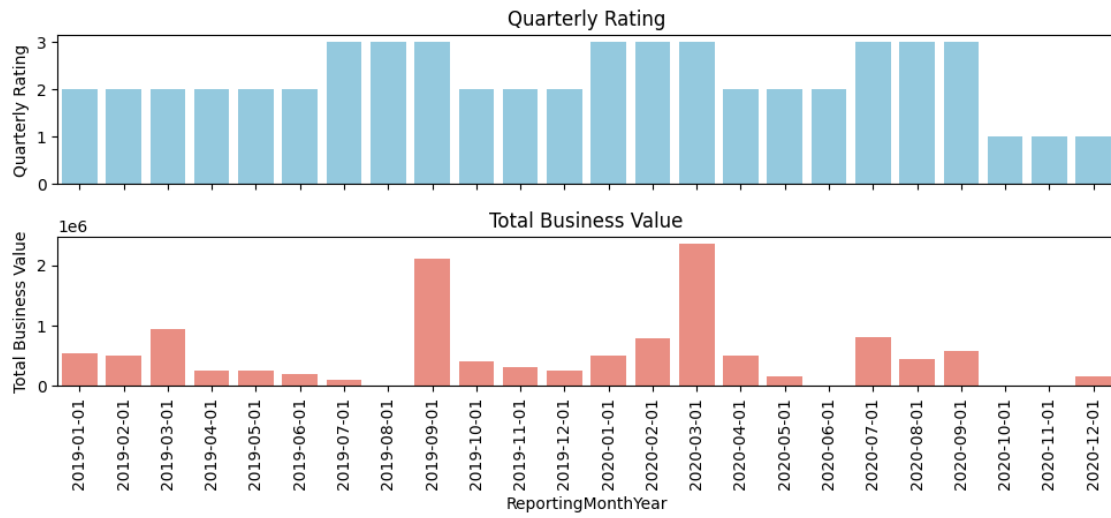
Driver ID: 173



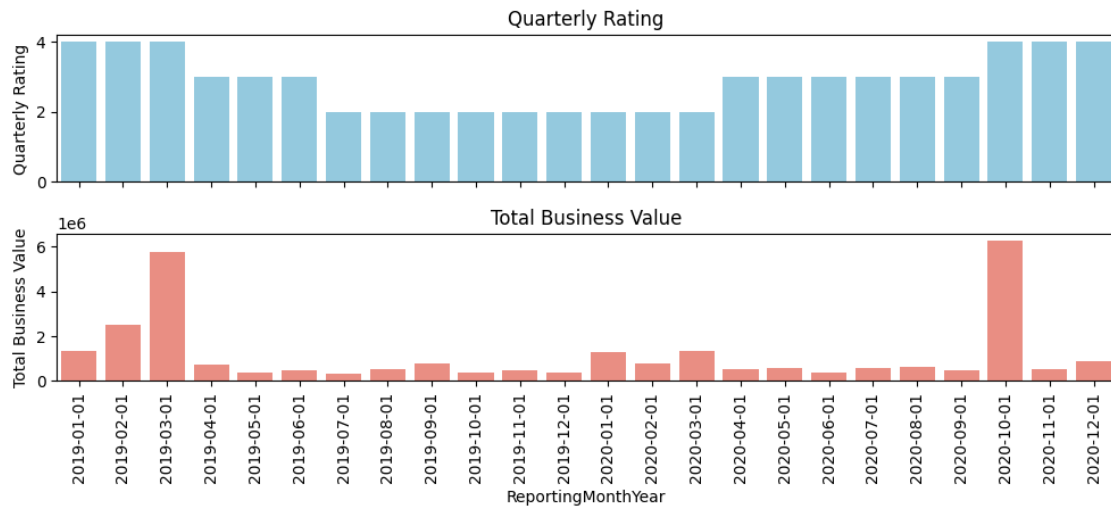
Driver ID: 199

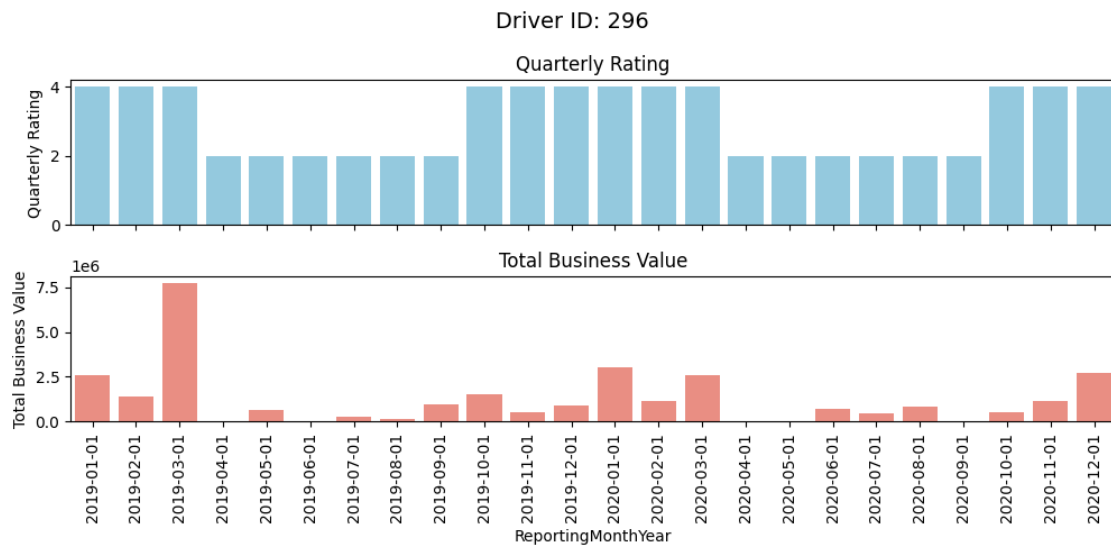
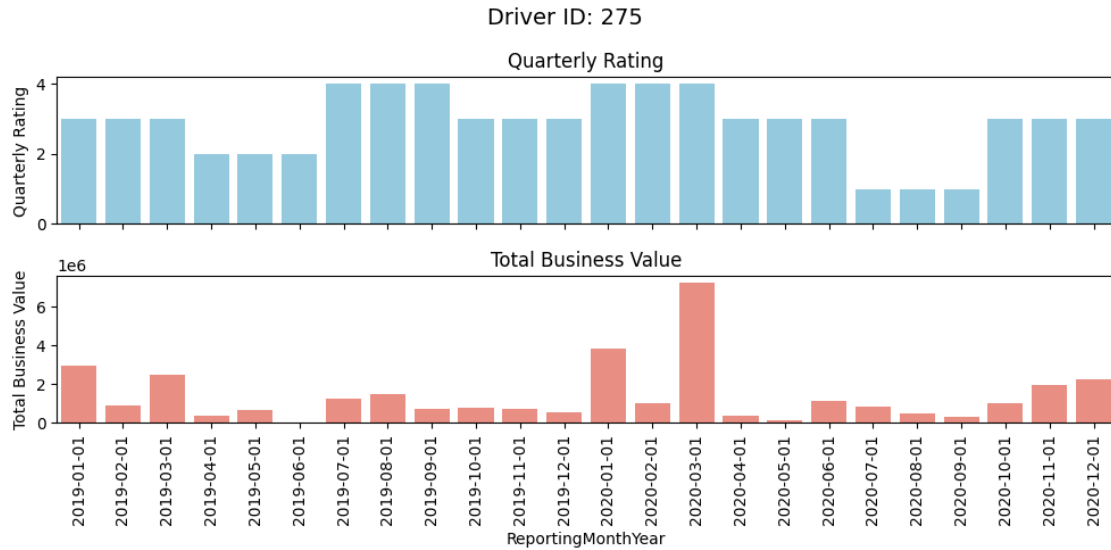


Driver ID: 213



Driver ID: 252





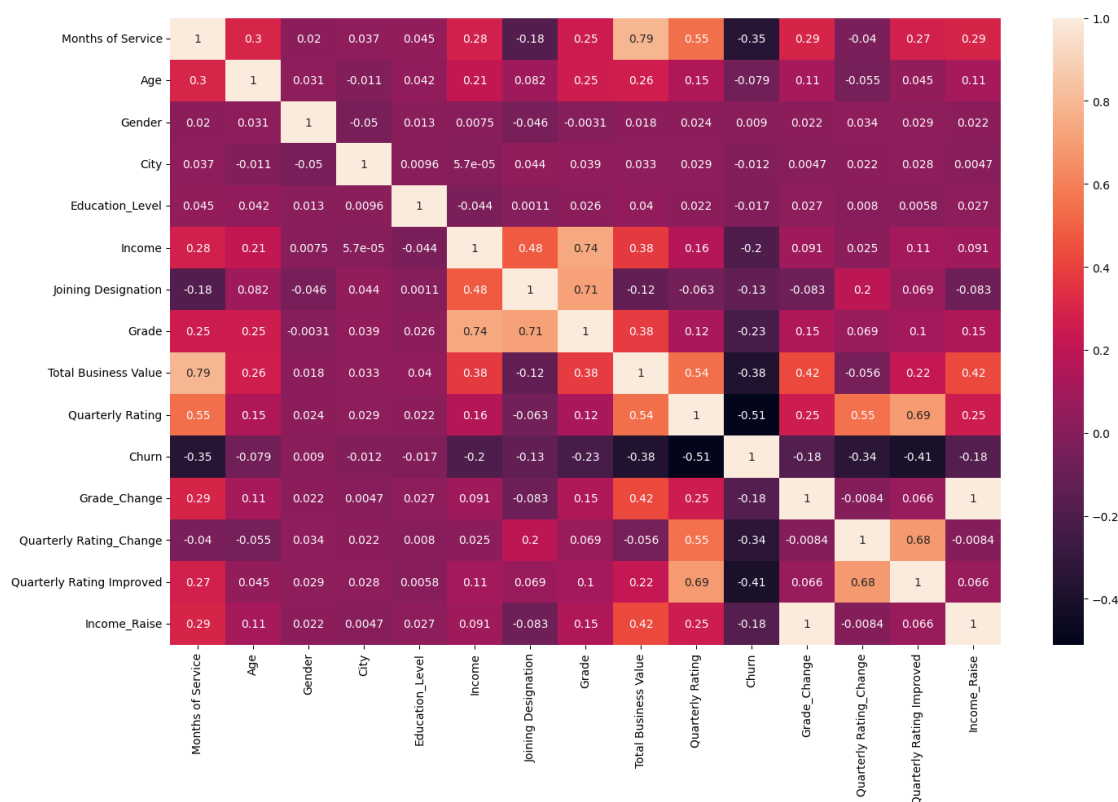
Insight

It can be observed that a significant drop in rating impacts the Total Business Value. Drop in rating demotivates the drivers, leading to accepting only a few rides or in somecases not accepting any rides and hence impacting the Total Business Value

6 Multivariate analysis

```
[282]: driver_df['Gender'].replace({'Male':0, 'Female':1}, inplace=True)
driver_df['Education_Level'].replace({'Graduate':0, '10+':1, '12+':2},
    ↪inplace=True)
driver_df['City'] = driver_df['City'].str[1:]

plt.figure(figsize=(15,10))
sns.heatmap(driver_df.drop(columns=['Driver_ID', 'Dateofjoining',
    ↪'LastWorkingDate', 'Income_Change']).corr(), annot=True)
plt.tight_layout()
plt.show()
```



Insight

Months of Service and Total Business Value are highly correlated

Income and Grade are highly correlated

Joining Designation and Grade are highly correlated

Quarterly Rating and Months of Service are highly correlated

Churn is decently correlated with Quarterly Rating, Total Business Value, Months of Service

7 Data Preprocessing

```
[283]: driver_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2381 entries, 0 to 2380
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Driver_ID                            2381 non-null   int64
1   Months of Service                    2381 non-null   int64
2   Age                                  2381 non-null   float64
3   Gender                              2381 non-null   category
4   City                                 2381 non-null   object
5   Education_Level                      2381 non-null   category
6   Income                              2381 non-null   int64
7   Dateofjoining                       2381 non-null   datetime64[ns]
8   LastWorkingDate                     1616 non-null   datetime64[ns]
9   Joining Designation                 2381 non-null   category
10  Grade                               2381 non-null   category
11  Total Business Value                 2381 non-null   int64
12  Quarterly Rating                     2381 non-null   int64
13  Churn                                2381 non-null   int64
14  Income_Change                        2381 non-null   int64
15  Grade_Change                         2381 non-null   int64
16  Quarterly Rating_Change               2381 non-null   int64
17  Quarterly Rating Improved             2381 non-null   int64
18  Income_Raise                         2381 non-null   int64
dtypes: category(4), datetime64[ns](2), float64(1), int64(11), object(1)
memory usage: 289.1+ KB
```

Insight

The columns Driver_ID, Gender, City, Education_Level, Dateofjoining, LastWorkingDate can be dropped as they do not contribute towards the driver churn rate

```
[284]: driver_df.drop(columns=['Driver_ID', 'Gender', 'City', 'Education_Level',
    ↪ 'Dateofjoining', 'LastWorkingDate', 'Income_Change'], inplace=True)
driver_df['Quarterly Rating'] = driver_df['Quarterly Rating'].astype('category')
driver_df['Churn'] = driver_df['Churn'].astype('category')
driver_df['Grade_Change'] = driver_df['Grade_Change'].astype('category')
driver_df['Quarterly Rating_Change'] = driver_df['Quarterly Rating_Change'].
    ↪ astype('category')
driver_df['Income_Raise'] = driver_df['Income_Raise'].astype('category')
driver_df.head()
```

```
[284]:   Months of Service   Age  Income  Joining Designation  Grade \
0                3  28.0    57387                1      1
```

1	2	31.0	67016	2	2
2	5	43.0	65603	2	2
3	3	29.0	46368	1	1
4	5	31.0	78728	3	3

	Total Business Value	Quarterly Rating	Churn	Grade_Change	\
0	1715580	2	1	0	
1	0	1	0	0	
2	350000	1	1	0	
3	120360	1	1	0	
4	1265000	2	0	0	

	Quarterly Rating_Change	Quarterly Rating Improved	Income_Raise
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	1	1	0

```
[285]: driver_df.duplicated().value_counts()
```

```
[285]: False      2381
      Name: count, dtype: int64
```

```
[286]: driver_df.isna().sum()
```

```
[286]: Months of Service      0
      Age                  0
      Income                0
      Joining Designation    0
      Grade                0
      Total Business Value   0
      Quarterly Rating       0
      Churn                 0
      Grade_Change          0
      Quarterly Rating_Change 0
      Quarterly Rating Improved 0
      Income_Raise          0
      dtype: int64
```

Outlier Treatment

```
[287]: def detect_outliers(df, method='iqr', threshold=1.5):
      """
      Detects outliers in a DataFrame using either 'iqr' or 'std' method.
      Returns a dictionary with column names and their outlier counts.
      """
```

```

numerical_cols = df.select_dtypes(include=np.number).columns
outlier_dict = {}

for col in numerical_cols:
    series = df[col]

    if method == 'iqr':
        q1 = series.quantile(0.25)
        q3 = series.quantile(0.75)
        iqr = q3 - q1
        lower = q1 - threshold * iqr
        upper = q3 + threshold * iqr
    elif method == 'std':
        mean = series.mean()
        std = series.std()
        lower = mean - threshold * std
        upper = mean + threshold * std
    else:
        raise ValueError("Method must be 'iqr' or 'std'.")

    lower_outliers = series[series < lower]
    upper_outliers = series[series > upper]
    outlier_dict[col] = {
        'lower_outliers': lower_outliers,
        'upper_outliers': upper_outliers,
        'total_outliers': len(lower_outliers) + len(upper_outliers),
        'upper_limit': upper,
        'lower_limit': lower
    }

return outlier_dict

```

IQR-based Outliers:

```

[288]: iqr_outliers = detect_outliers(driver_df, method='iqr')
for col, out in iqr_outliers.items():
    print(f"[IQR] Column '{col}' has {out['total_outliers']} outliers")

```

```

[IQR] Column 'Months of Service' has 249 outliers
[IQR] Column 'Age' has 25 outliers
[IQR] Column 'Income' has 48 outliers
[IQR] Column 'Total Business Value' has 336 outliers
[IQR] Column 'Quarterly Rating Improved' has 358 outliers

```

Std-based Outliers:

```

[289]: std_outliers = detect_outliers(driver_df, method='std', threshold=3)
for col, out in std_outliers.items():

```

```
print(f"[STD] Column '{col}' has {out['total_outliers']} outliers")
```

```
[STD] Column 'Months of Service' has 0 outliers
[STD] Column 'Age' has 14 outliers
[STD] Column 'Income' has 16 outliers
[STD] Column 'Total Business Value' has 64 outliers
[STD] Column 'Quarterly Rating Improved' has 0 outliers
```

Capping Outliers

```
[290]: # Cap Total Business Value to upper limit from STD method
cap_value = std_outliers['Total Business Value']['upper_limit']
driver_df['Total Business Value'] = driver_df['Total Business Value'].
↳clip(upper=cap_value)
```

**** Multicollinearity Check ****

```
[295]: features_df = driver_df.drop(columns=['Churn']) # Drop target column
features_df = features_df.drop(columns=features_df.
↳select_dtypes(include='category').columns) # Drop category columns
features_df = sm.add_constant(features_df) # Adding a constant column for the
↳intercept
vif_df = pd.DataFrame()
vif_df['Features'] = features_df.columns
vif_df['VIF'] = [variance_inflation_factor(features_df.values, idx) for idx in
↳range(len(features_df.columns))]
vif_df['VIF'] = round(vif_df['VIF'], 2)
vif_df = vif_df.sort_values(by='VIF', ascending=False)
vif_df
```

```
[295]:
```

	Features	VIF
0	const	36.42
4	Total Business Value	3.99
1	Months of Service	3.83
3	Income	1.19
2	Age	1.12
5	Quarterly Rating Improved	1.09

Insight

Based on the above VIF scores, it can be concluded that there are no multicollinear numerical features

Encode categorical variables

```
[296]: final_df = driver_df.copy()
```

```
[297]: X = final_df.drop(columns=['Churn'])
y = final_df['Churn']
```

```
X.shape, y.shape
```

```
[297]: ((2381, 11), (2381,))
```

```
[298]: # Encode target variable
y = y.astype(int)
```

```
[299]: # Encode features with just 2 classes as 0 or 1
X[['Grade_Change', 'Quarterly Rating_Change', 'Income_Raise']] =
↳X[['Grade_Change', 'Quarterly Rating_Change', 'Income_Raise']].astype('int8')
```

```
[300]: X.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2381 entries, 0 to 2380
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Months of Service                    2381 non-null   int64
1   Age                                  2381 non-null   float64
2   Income                              2381 non-null   int64
3   Joining Designation                 2381 non-null   category
4   Grade                               2381 non-null   category
5   Total Business Value                 2381 non-null   float64
6   Quarterly Rating                    2381 non-null   category
7   Grade_Change                        2381 non-null   int8
8   Quarterly Rating_Change             2381 non-null   int8
9   Quarterly Rating Improved            2381 non-null   int64
10  Income_Raise                        2381 non-null   int8
dtypes: category(3), float64(2), int64(3), int8(3)
memory usage: 107.7 KB
```

```
[301]: categorical_columns = X.select_dtypes(include='category').columns
categorical_columns
```

```
[301]: Index(['Joining Designation', 'Grade', 'Quarterly Rating'], dtype='object')
```

```
[304]: encoder = OneHotEncoder(sparse_output=False)
encoded_data = encoder.fit_transform(X[categorical_columns])
encoded_df = pd.DataFrame(encoded_data, columns = encoder.
↳get_feature_names_out(categorical_columns))
X = pd.concat([X, encoded_df], axis=1)
X.drop(columns = categorical_columns, inplace=True)
X.head()
```

```
[304]:   Months of Service   Age  Income  Total Business Value  Grade_Change \
0                3  28.0    57387             1715580.0           0
```


1	2	31.0	67016	0.0	0
2	5	43.0	65603	350000.0	0
3	3	29.0	46368	120360.0	0
4	5	31.0	78728	1265000.0	0

	Quarterly Rating_Change	Quarterly Rating Improved	Income_Raise	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	1	1	0	

	Joining Designation_1	Joining Designation_2	...	Joining Designation_5	\
0	1.0	0.0	...	0.0	
1	0.0	1.0	...	0.0	
2	0.0	1.0	...	0.0	
3	1.0	0.0	...	0.0	
4	0.0	0.0	...	0.0	

	Grade_1	Grade_2	Grade_3	Grade_4	Grade_5	Quarterly Rating_1	\
0	1.0	0.0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	0.0	1.0	
2	0.0	1.0	0.0	0.0	0.0	1.0	
3	1.0	0.0	0.0	0.0	0.0	1.0	
4	0.0	0.0	1.0	0.0	0.0	0.0	

	Quarterly Rating_2	Quarterly Rating_3	Quarterly Rating_4
0	1.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	1.0	0.0	0.0

[5 rows x 22 columns]

Model building

Train-test split

```
[305]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=0)
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
[305]: ((1904, 22), (477, 22), (1904,), (477,))
```

Data Normalization / Standardization

```
[306]: min_max_scaler = MinMaxScaler()
# Fit min_max_scaler to training data
min_max_scaler.fit(X_train)
# Scale the training and testing data
X_train = pd.DataFrame(min_max_scaler.transform(X_train), columns=X_train.
↳columns)
X_test = pd.DataFrame(min_max_scaler.transform(X_test), columns=X_test.columns)
```

```
[307]: X_train.head()
```

```
[307]:
```

	Months of Service	Age	Income	Total Business Value	Grade_Change	\
0	0.260870	0.205882	0.261253	0.074297	0.0	
1	0.000000	0.205882	0.176872	0.041541	0.0	
2	0.000000	0.235294	0.187099	0.041541	0.0	
3	0.260870	0.558824	0.275166	0.119183	0.0	
4	0.217391	0.117647	0.103028	0.106669	0.0	

	Quarterly Rating_Change	Quarterly Rating Improved	Income_Raise	\
0	0.5	0.0	0.0	
1	0.5	0.0	0.0	
2	0.5	0.0	0.0	
3	0.5	0.0	0.0	
4	0.5	0.0	0.0	

	Joining Designation_1	Joining Designation_2	...	Joining Designation_5	\
0	0.0	1.0	...	0.0	
1	1.0	0.0	...	0.0	
2	1.0	0.0	...	0.0	
3	0.0	0.0	...	0.0	
4	1.0	0.0	...	0.0	

	Grade_1	Grade_2	Grade_3	Grade_4	Grade_5	Quarterly Rating_1	\
0	0.0	1.0	0.0	0.0	0.0	1.0	
1	1.0	0.0	0.0	0.0	0.0	1.0	
2	1.0	0.0	0.0	0.0	0.0	1.0	
3	0.0	0.0	1.0	0.0	0.0	1.0	
4	1.0	0.0	0.0	0.0	0.0	1.0	

	Quarterly Rating_2	Quarterly Rating_3	Quarterly Rating_4
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0


```
[5 rows x 22 columns]
```

Check for imbalance in target class

```
[308]: y_train.value_counts(normalize=True)*100
```

```
[308]: Churn
      1    68.644958
      0    31.355042
      Name: proportion, dtype: float64
```

We can see a clear imbalance in the target class with 1 being ~69% and 0 being ~31%. Hence, I will use SMOTE to fix this imbalance

```
[309]: sm = SMOTE(random_state=0)
      X_train, y_train = sm.fit_resample(X_train, y_train)
      y_train.value_counts(normalize=True)*100
```

```
[309]: Churn
      1    50.0
      0    50.0
      Name: proportion, dtype: float64
```

8 *Ensemble Learning: Bagging - RandomForestClassifier*

```
[310]: from sklearn.model_selection import RandomizedSearchCV
      from sklearn.ensemble import RandomForestClassifier
      from scipy.stats import randint

      # Define parameter distribution (random ranges instead of full grid)
      param_dist = {
          'n_estimators': randint(100, 1000),
          'max_features': ['sqrt', 'log2'],
          'max_depth': randint(10, 100),
          'min_samples_split': randint(2, 10)
      }

      # Initialize classifier and RandomizedSearchCV
      rf = RandomForestClassifier(random_state=42)
      rf_random = RandomizedSearchCV(
          estimator=rf,
          param_distributions=param_dist,
          n_iter=50,                # Number of random combinations to try
          cv=3,
          verbose=1,
          n_jobs=-1,
          random_state=42
      )
```

```
# Fit the model
rf_random.fit(X_train, y_train)

# Evaluate best parameters
print("Best parameters found: ", rf_random.best_params_)
```

Fitting 3 folds for each of 50 candidates, totalling 150 fits
 Best parameters found: {'max_depth': 99, 'max_features': 'sqrt',
 'min_samples_split': 3, 'n_estimators': 439}

```
[311]: def evaluate_model(model, X, y, dataset_name="Dataset"):
        y_pred = model.predict(X)
        print(f"\033[91m\033[1m{dataset_name}:\033[0m")
        print(f"Accuracy: {accuracy_score(y, y_pred):.4f}")
        print("Classification Report:\n", classification_report(y, y_pred))

# Evaluate on both train and test sets
evaluate_model(rf_random, X_train, y_train, "Train Data")
evaluate_model(rf_random, X_test, y_test, "Test Data")
```

Train Data:

Accuracy: 0.9996

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1307
1	1.00	1.00	1.00	1307
accuracy			1.00	2614
macro avg	1.00	1.00	1.00	2614
weighted avg	1.00	1.00	1.00	2614

Test Data:

Accuracy: 0.7778

Classification Report:

	precision	recall	f1-score	support
0	0.70	0.64	0.67	168
1	0.81	0.85	0.83	309
accuracy			0.78	477
macro avg	0.76	0.75	0.75	477
weighted avg	0.77	0.78	0.78	477

Insight

The training accuracy is 1 whereas testing accuracy is 0.778. This is a case of overfitting.

The best parameters found are well within the provided range

Feature Importance Plot

```
[312]: def plot_feature_importance(estimator, feature_names, top_n=None,
    importance_attr='feature_importances_',
    figsize=(10, 6), color="royalblue", title="Feature_
    Importance", grid=True):
    """
    Plots feature importances for models with attribute like
    'feature_importances_' or 'coef_'.

    Parameters:
    - estimator: Trained model
    - feature_names: List of feature names
    - top_n: Top N features to plot (default: all)
    - importance_attr: Attribute to extract importance ('feature_importances_'
    or 'coef_')
    - figsize: Size of the plot
    - color: Bar color
    - title: Title of the plot
    - grid: Whether to show gridlines
    """

    # Check attribute availability
    if not hasattr(estimator, importance_attr):
        raise ValueError(f"The estimator does not have '{importance_attr}'
    attribute.")

    # Extract importances
    importances = getattr(estimator, importance_attr)
    if isinstance(importances, np.ndarray) and importances.ndim > 1:
        importances = importances[0] # For multiclass linear models

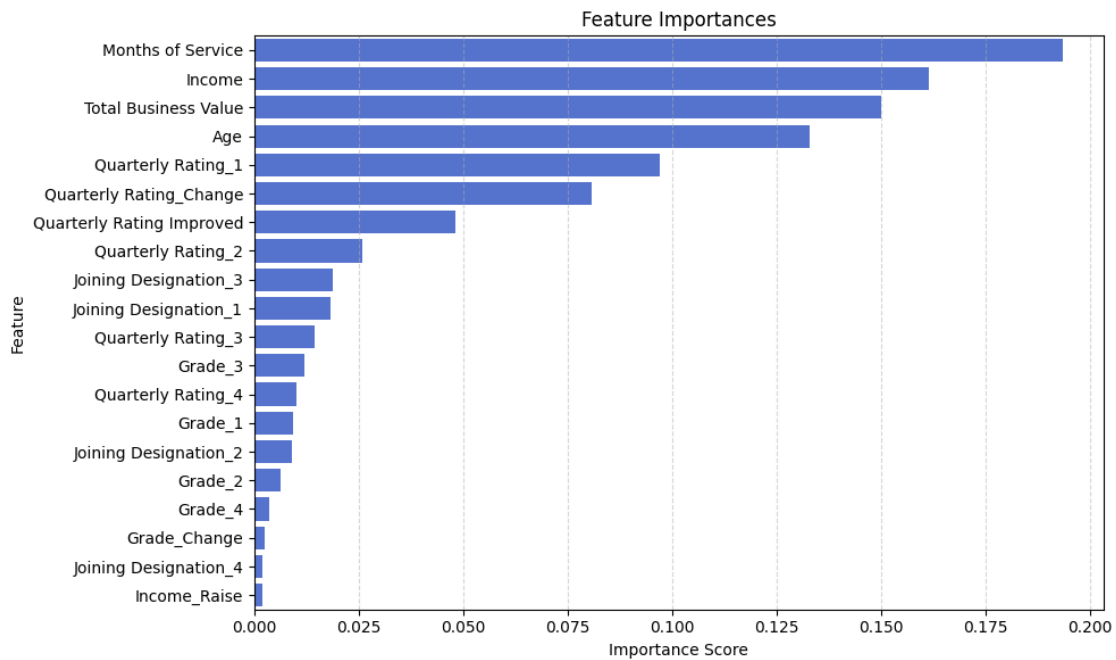
    feature_df = pd.DataFrame({
        'Feature': feature_names,
        'Importance': np.abs(importances) # Take absolute for 'coef_'-based
    models
    }).sort_values(by='Importance', ascending=False)

    if top_n:
        feature_df = feature_df.head(top_n)

    # Plotting
    plt.figure(figsize=figsize)
    sns.barplot(data=feature_df, x='Importance', y='Feature', color=color)
    plt.title(title)
```

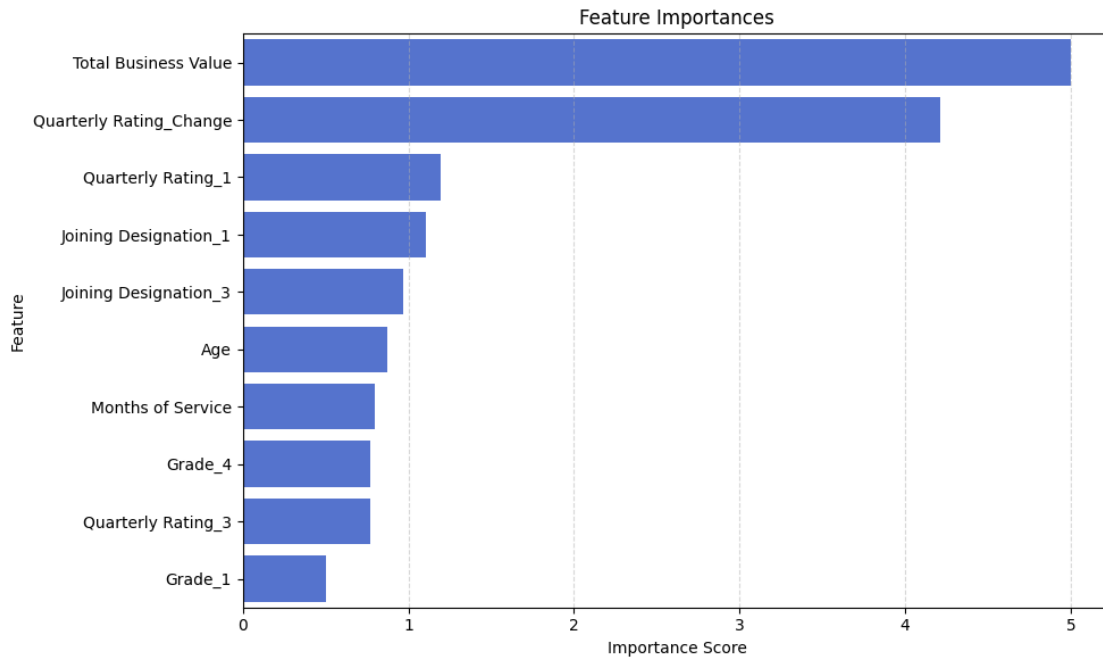
```
plt.xlabel('Importance Score')
plt.ylabel('Feature')
if grid:
    plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

```
plot_feature_importance(rf_random.best_estimator_, X_train.columns, top_n=20)
```



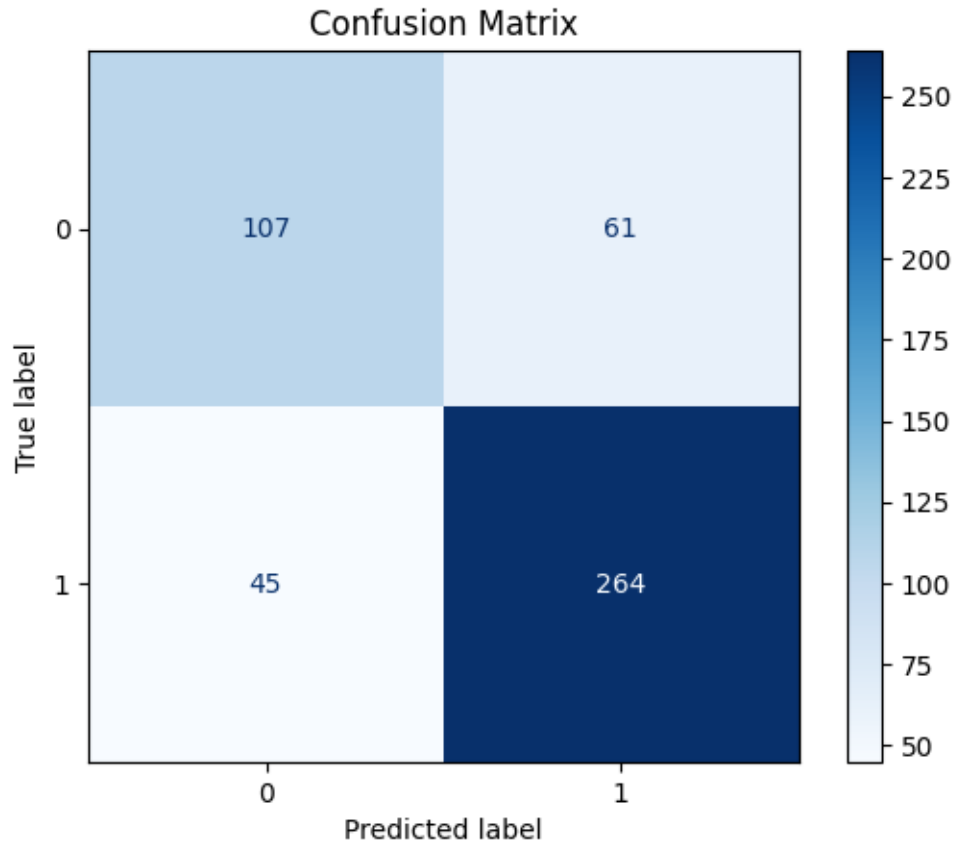
```
[317]: # Fit logistic regression
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train, y_train)

# Plot feature importance using coef_
plot_feature_importance(logreg, X_train.columns, top_n=10,
    importance_attr='coef_')
```



9 Confusion Matrix

```
[321]: def display_confusion_matrix(y_test, y_pred):  
    # Compute confusion matrix  
    cm = confusion_matrix(y_test, y_pred)  
  
    # Plot confusion matrix  
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)  
    disp.plot(cmap=plt.cm.Blues)  
    plt.title('Confusion Matrix')  
    plt.show()  
    y_pred = rf_random.predict(X_test)  
    display_confusion_matrix(y_test, y_pred)
```



ROC Curve

```
[326]: def plot_multiclass_roc(estimator, X_train, X_test, y_train, y_test,
    ↪class_names=None, average='micro'):
    # Binarize the output
    y_test_binarized = label_binarize(y_test, classes=[0, 1, 2])
    n_classes = y_test_binarized.shape[1]-1

    # Compute ROC curve and ROC area for each class
    classifier = OneVsRestClassifier(estimator)
    y_score = classifier.fit(X_train, y_train).predict_proba(X_test)

    fpr = dict()
    tpr = dict()
    roc_auc = dict()
    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_test_binarized[:, i], y_score[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Plot ROC curve for each class
```

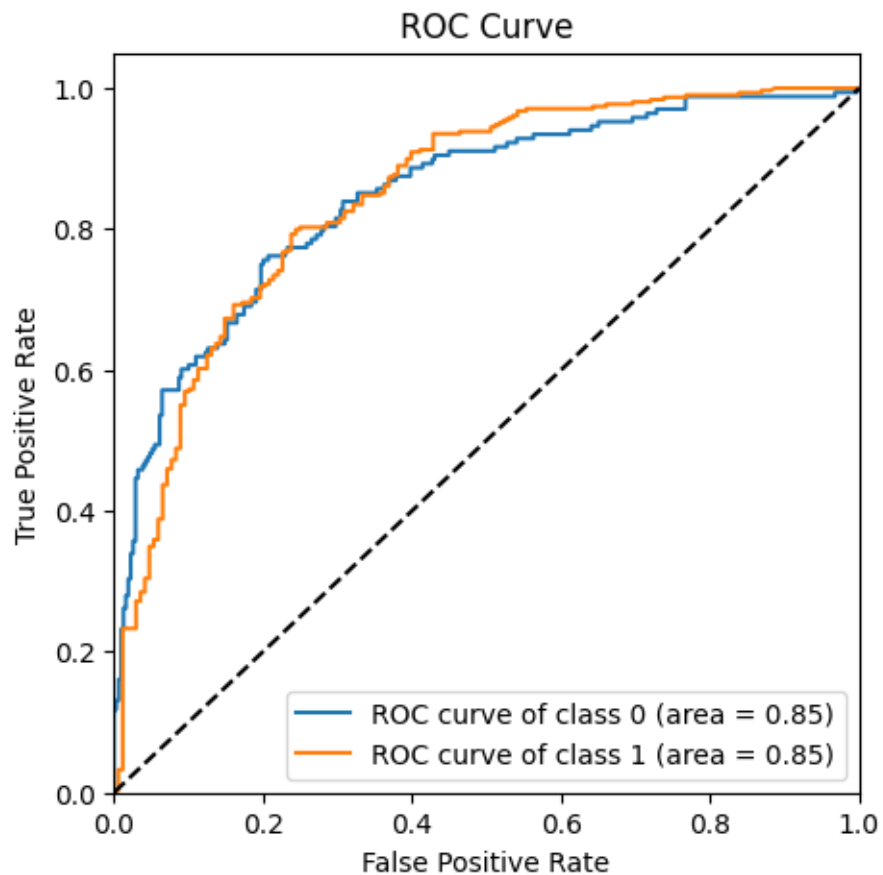


```

plt.figure(figsize=(5, 5))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label='ROC curve of class {0} (area = {1:0.
↪2f})'.format(i, roc_auc[i]))
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc='lower right')
plt.show()

plot_multiclass_roc(rf_random.best_estimator_, X_train, X_test, y_train, y_test)

```



Precision-Recall Curve

```
[329]: def plot_pr_curve(estimator, X_train, X_test, y_train, y_test):
    # Binarize the output
    y_test_binarized = label_binarize(y_test, classes=[0, 1, 2])
    n_classes = y_test_binarized.shape[1]-1

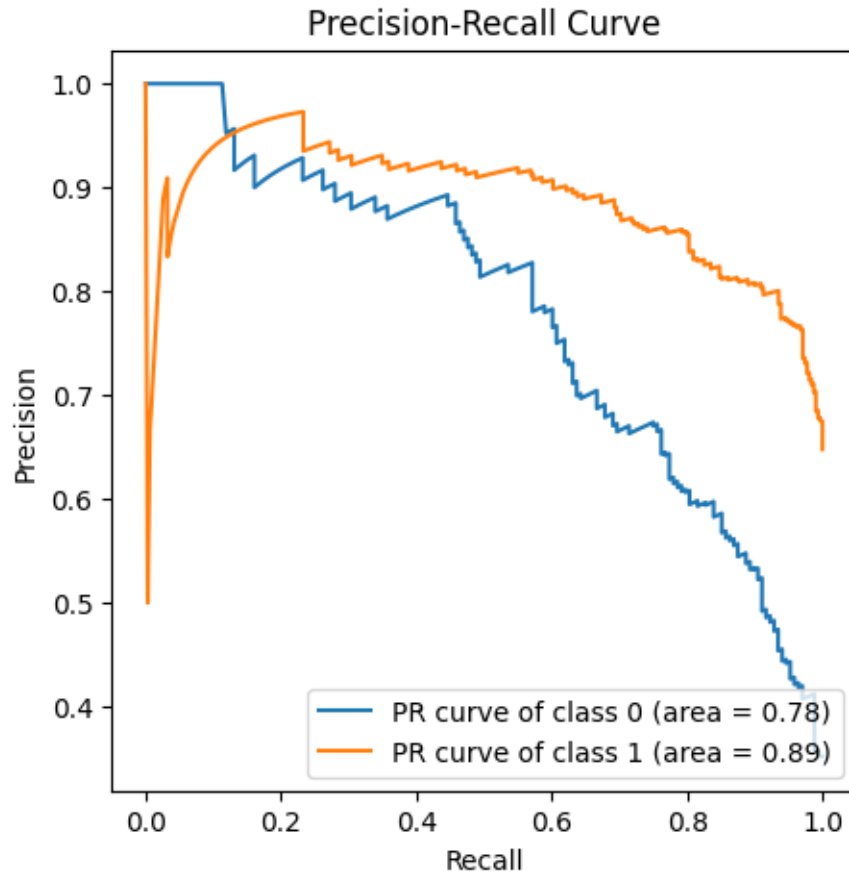
    # Compute ROC curve and ROC area for each class
    classifier = OneVsRestClassifier(estimator)
    y_score = classifier.fit(X_train, y_train).predict_proba(X_test)

    # For each class
    precision = dict()
    recall = dict()
    average_precision = dict()
    for i in range(n_classes):
        precision[i], recall[i], _ = precision_recall_curve(y_test_binarized[:, i],
↪ y_score[:, i])
        average_precision[i] = average_precision_score(y_test_binarized[:, i],
↪ y_score[:, i])

    # Plot Precision-Recall curve for each class
    plt.figure(figsize=(5, 5))
    for i in range(n_classes):
        plt.plot(recall[i], precision[i], label='PR curve of class {0} (area =
↪ {1:0.2f})'.format(i, average_precision[i]))

    plt.xlabel('Recall')
    plt.ylabel('Precision')
    plt.title('Precision-Recall Curve')
    plt.legend(loc='lower right')
    plt.show()

plot_pr_curve(rf_random.best_estimator_, X_train, X_test, y_train, y_test)
```



10 Ensemble Learning: Boosting - GradientBoostingClassifier

```
[333]: # Define parameter grid (reduced range for faster search)
param_grid = {
    'n_estimators': np.linspace(100, 500, 5, dtype=int),          # fewer
    ↳ steps
    'learning_rate': np.logspace(-2, 0, 5),                      # 0.01
    ↳ to 1.0
    'max_depth': np.arange(3, 8),                                #
    ↳ commonly 3-7 is sufficient
    'min_samples_split': [2, 5, 10],                             #
    ↳ practical defaults
    'min_samples_leaf': [1, 3, 5],                               #
    ↳ prevent overfitting
    'subsample': [0.6, 0.8, 1.0]                                 #
    ↳ standard values
}
```

```

# Initialize model and RandomizedSearchCV
gb = GradientBoostingClassifier(random_state=42)
gb_random = RandomizedSearchCV(
    estimator=gb,
    param_distributions=param_grid,
    n_iter=50,                      # reduced from 300 + faster, still broad
    ↪coverage
    scoring='accuracy',            # or 'f1_weighted' / 'roc_auc_ovr' for multiclass
    cv=5,                          # 5-fold CV for stability
    verbose=1,
    random_state=42,
    n_jobs=-1
)

# Fit the model
gb_random.fit(X_train, y_train)

# Output best parameters
print(" Best parameters for GradientBoostingClassifier:\n", gb_random.
    ↪best_params_)

```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

```

Best parameters for GradientBoostingClassifier:
{'subsample': 0.8, 'n_estimators': np.int64(500), 'min_samples_split': 2,
'min_samples_leaf': 1, 'max_depth': np.int64(7), 'learning_rate':
np.float64(0.03162277660168379)}

```

```

[335]: from sklearn.metrics import accuracy_score, classification_report

# Terminal color formatting
RED_BOLD = '\033[91m\033[1m'
RESET = '\033[0m'

def evaluate_model(model, X_train, y_train, X_test, y_test, title="Model_
    ↪Evaluation"):
    print(f"{RED_BOLD}{title} - Train Data:{RESET}")
    y_pred_train = model.predict(X_train)
    print("Accuracy:", accuracy_score(y_train, y_pred_train))
    print("Classification Report:\n", classification_report(y_train,
    ↪y_pred_train))

    print(f"\n{RED_BOLD}{title} - Test Data:{RESET}")
    y_pred_test = model.predict(X_test)
    print("Accuracy:", accuracy_score(y_test, y_pred_test))
    print("Classification Report:\n", classification_report(y_test,
    ↪y_pred_test))

```

```
evaluate_model.gb_random.best_estimator_, X_train, y_train, X_test, y_test,
↳title="GradientBoostingClassifier")
```

GradientBoostingClassifier - Train Data:

Accuracy: 0.9996174445294568

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1307
1	1.00	1.00	1.00	1307
accuracy			1.00	2614
macro avg	1.00	1.00	1.00	2614
weighted avg	1.00	1.00	1.00	2614

GradientBoostingClassifier - Test Data:

Accuracy: 0.7819706498951782

Classification Report:

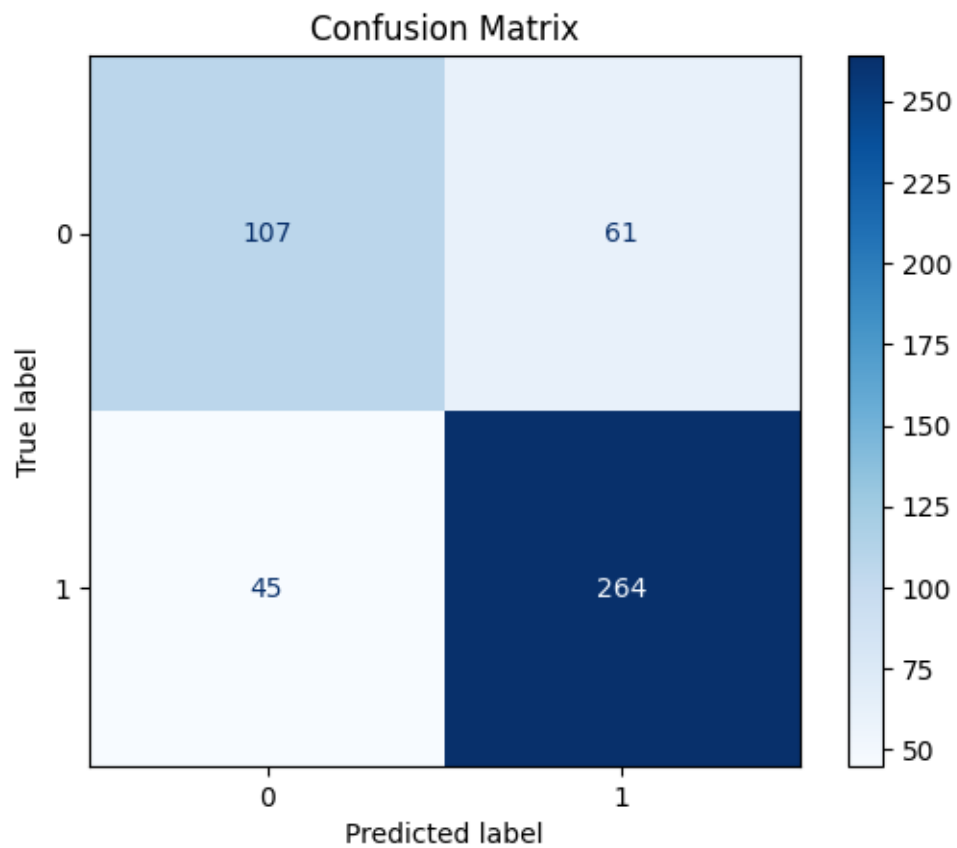
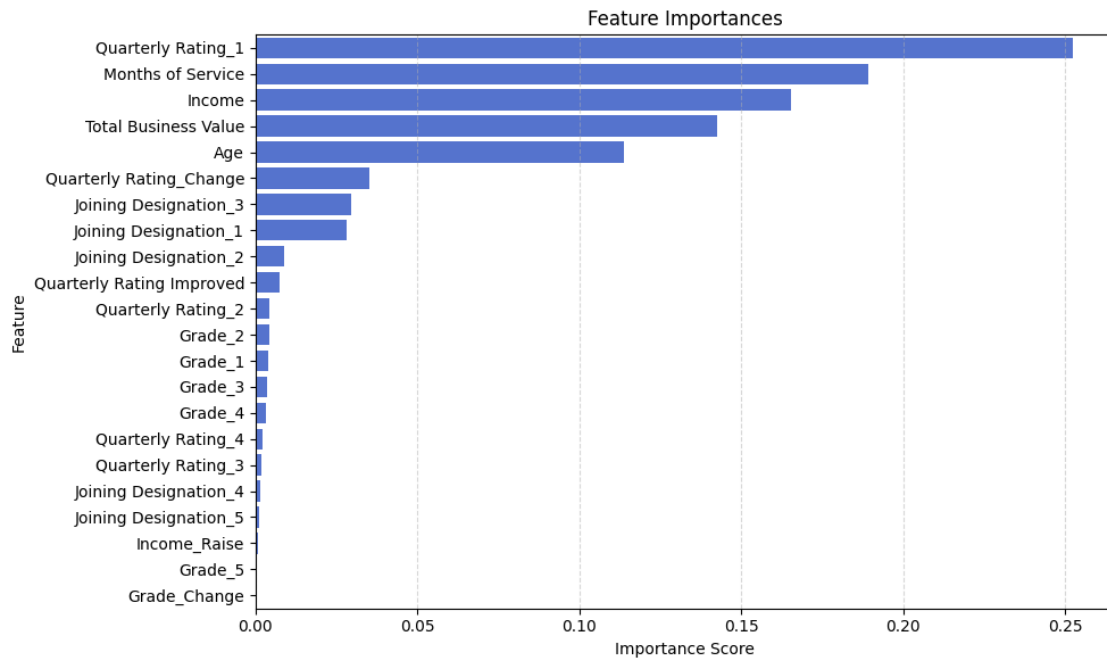
	precision	recall	f1-score	support
0	0.72	0.62	0.67	168
1	0.81	0.87	0.84	309
accuracy			0.78	477
macro avg	0.77	0.74	0.75	477
weighted avg	0.78	0.78	0.78	477

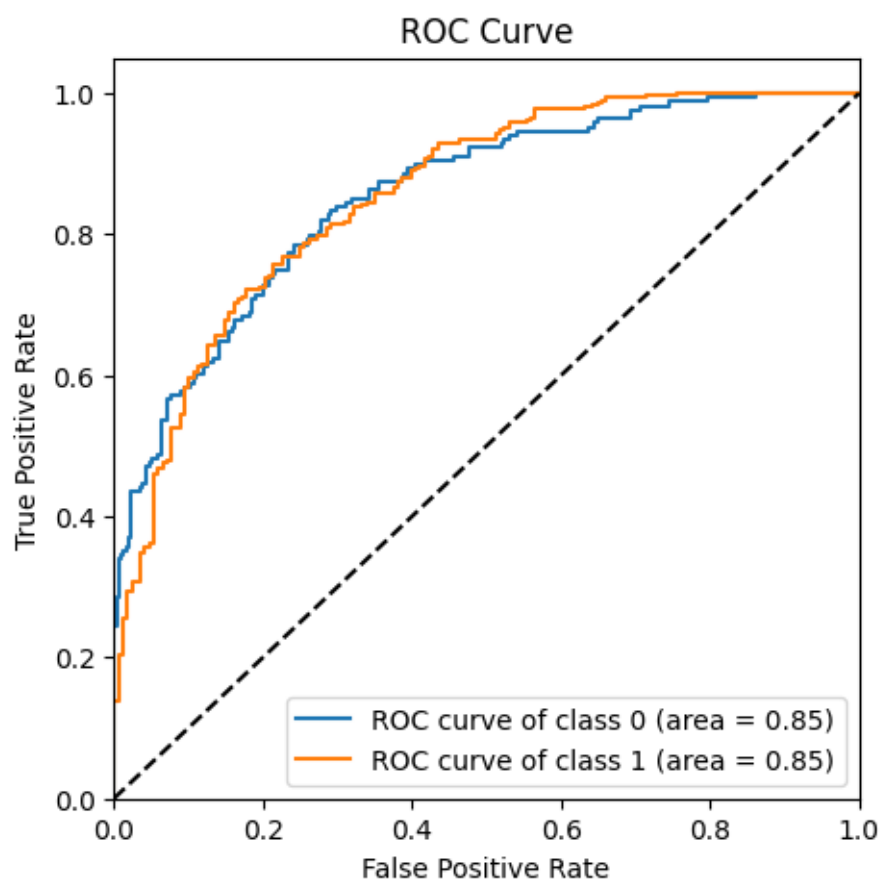
Insight

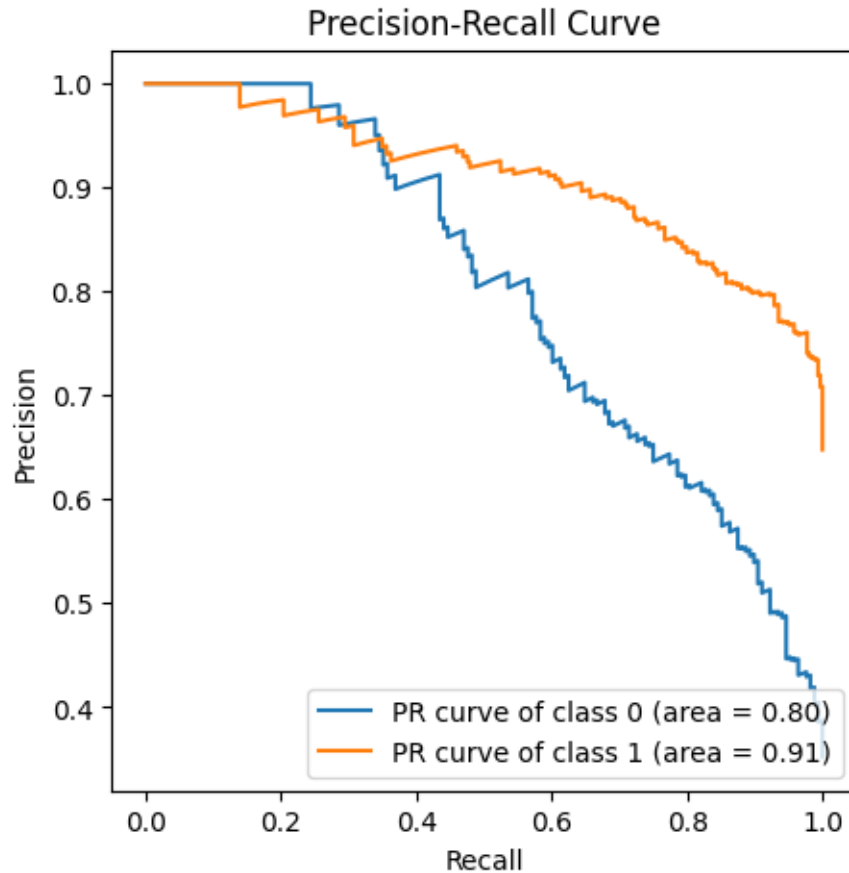
The training accuracy is 1 whereas testing accuracy is 0.786. This is also a case of overfitting.

Performance

```
[336]: plot_feature_importance.gb_random.best_estimator_, X_train.columns)
display_confusion_matrix(y_test, y_pred)
plot_multiclass_roc.gb_random.best_estimator_, X_train, X_test, y_train, y_test)
plot_pr_curve.gb_random.best_estimator_, X_train, X_test, y_train, y_test)
```







Insight The top 5 features as per the GradientBoostingClassifier are

–Quarterly Rating 1

–Months of Service

–Income

–Total Business Value

–Age Both the classes 0 and 1 have a decent Area Under the ROC curve of 0.85 The Area Under the PR curve for class 0 is 0.79 and class 1 is 0.91

11 Ensemble Learning: Boosting - XGBClassifier

```
[339]: param_grid = {
    'n_estimators': np.arange(100, 1001, 100),
    'learning_rate': np.logspace(-3, 0, 10),
    'max_depth': np.arange(3, 11),
    'min_child_weight': np.arange(1, 11),
    'gamma': np.logspace(-3, 1, 10),
```



```

    'subsample': np.linspace(0.5, 1.0, 6),
    'colsample_bytree': np.linspace(0.5, 1.0, 6)
}

# Initialize the classifier
xgb = XGBClassifier(eval_metric='mlogloss', use_label_encoder=False)

# Set up RandomizedSearchCV
xgb_random = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_grid,
    n_iter=100, # reduced from 300 for speed; increase if needed
    cv=3,
    verbose=1, # more moderate verbosity
    random_state=42,
    n_jobs=-1,
    scoring='accuracy' # you can also use 'f1_weighted' or others
)

# Fit the model to training data
xgb_random.fit(X_train, y_train)

# Output best parameters
print("\033[1mBest parameters found for XGBoost:\033[0m", xgb_random.
↪best_params_)

```

Fitting 3 folds for each of 100 candidates, totalling 300 fits
 Best parameters found for XGBoost: {'subsample': np.float64(0.5),
 'n_estimators': np.int64(200), 'min_child_weight': np.int64(1), 'max_depth':
 np.int64(7), 'learning_rate': np.float64(0.046415888336127774), 'gamma':
 np.float64(0.021544346900318832), 'colsample_bytree': np.float64(1.0)}

```

[341]: def evaluate_model(model, X_train, y_train, X_test, y_test, title="Model"):
    color = '\033[91m'
    bold = '\033[1m'
    end = '\033[0m'

    # Train evaluation
    y_train_pred = model.predict(X_train)
    print(f"{color}{bold}{title} - Train Data:{end}")
    print("Accuracy:", accuracy_score(y_train, y_train_pred))
    print("Classification Report:\n", classification_report(y_train,
↪y_train_pred))

    # Test evaluation
    y_test_pred = model.predict(X_test)
    print(f"{color}{bold}{title} - Test Data:{end}")

```

```

print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Classification Report:\n", classification_report(y_test,
↪y_test_pred))

evaluate_model(xgb_random.best_estimator_, X_train, y_train, X_test, y_test,
↪title="XGBoost Classifier")

```

XGBoost Classifier - Train Data:

Accuracy: 0.9456771231828616

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.94	0.95	1307
1	0.94	0.96	0.95	1307
accuracy			0.95	2614
macro avg	0.95	0.95	0.95	2614
weighted avg	0.95	0.95	0.95	2614

XGBoost Classifier - Test Data:

Accuracy: 0.8071278825995807

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.67	0.71	168
1	0.83	0.88	0.86	309
accuracy			0.81	477
macro avg	0.79	0.78	0.78	477
weighted avg	0.80	0.81	0.80	477

Insight

The training accuracy has reduced to 0.939 whereas testing accuracy has slightly increased to 0.813. This is still a case of overfitting but better than all the previous models.

This model is also faster than the previous models

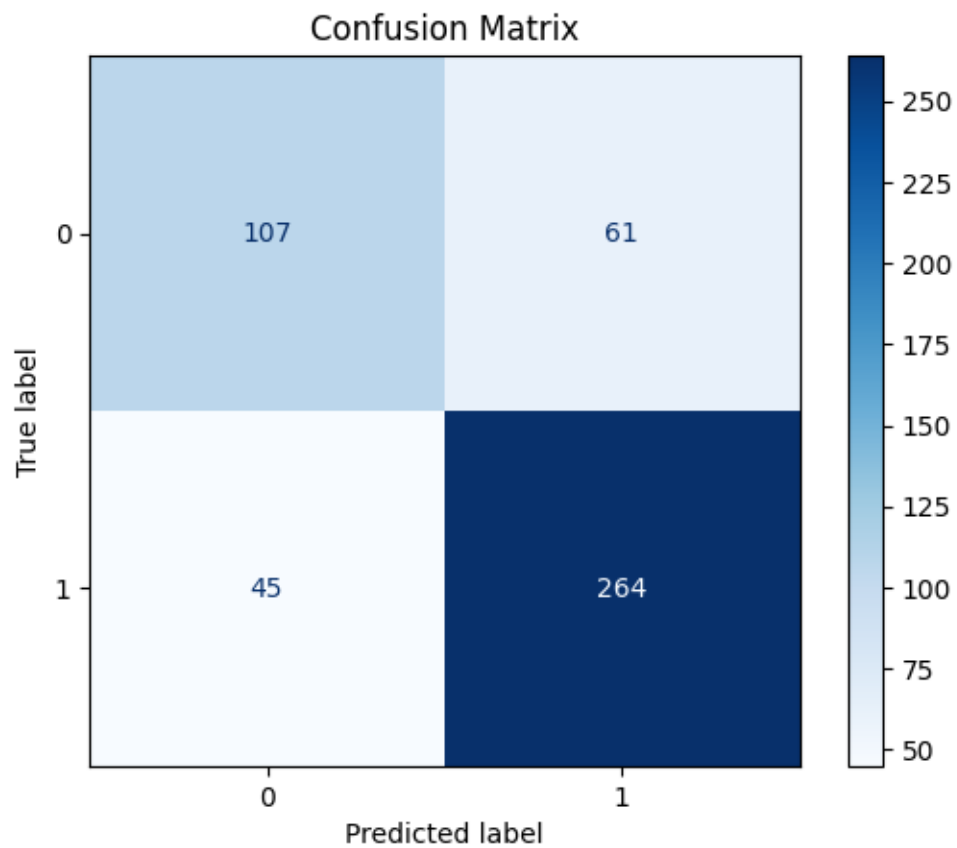
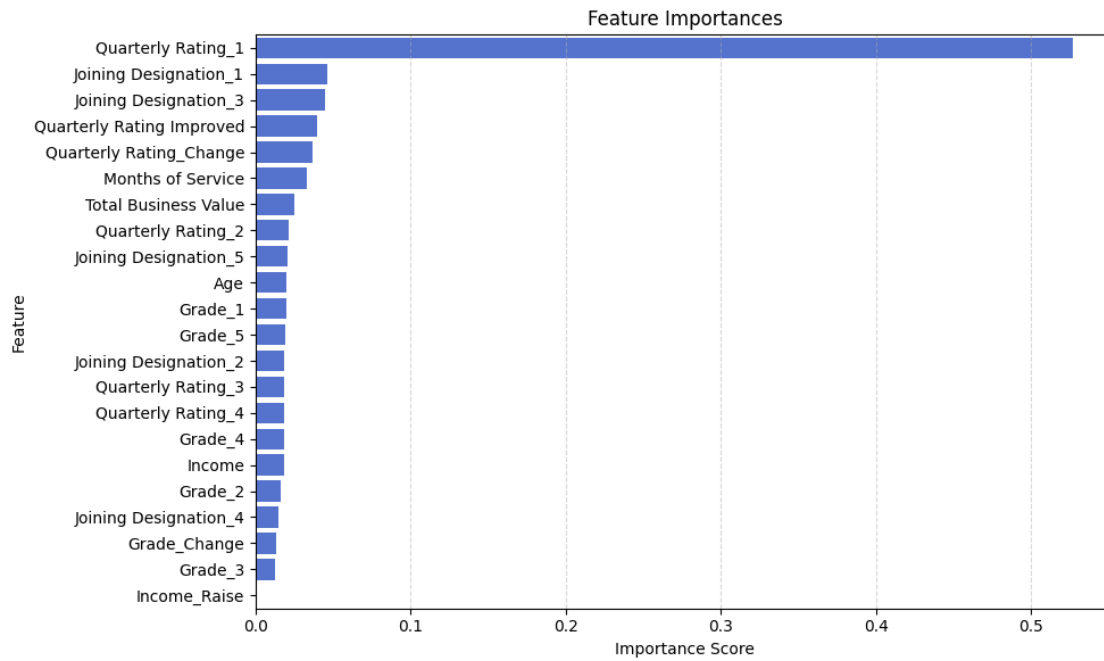
Performance

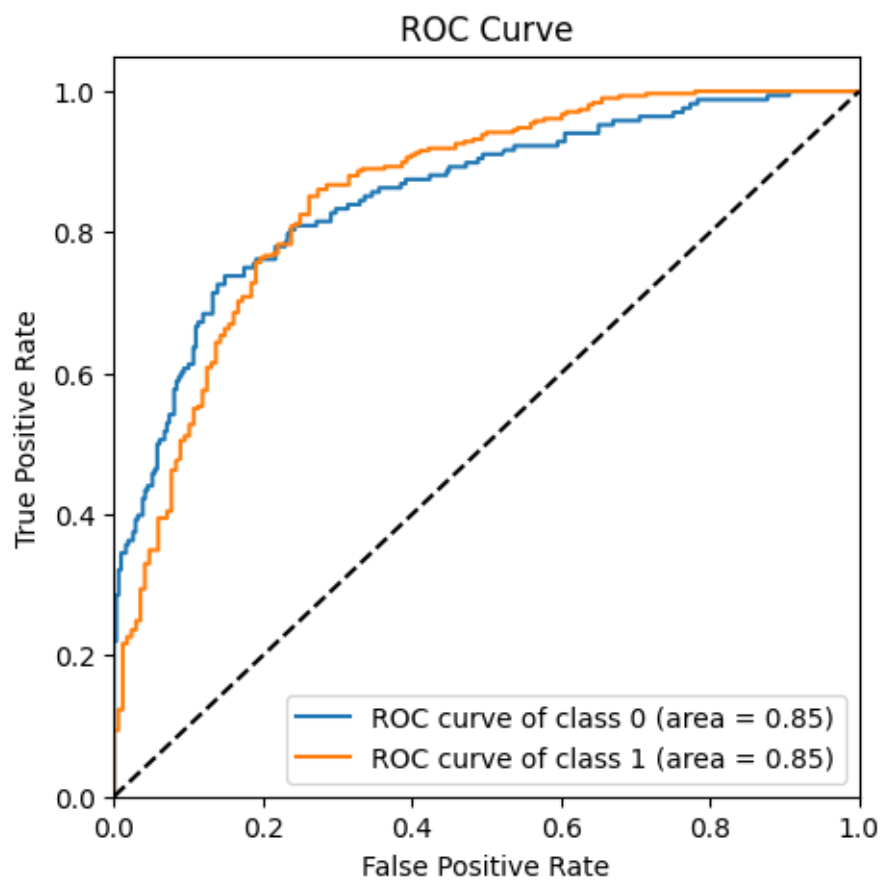
[342]:

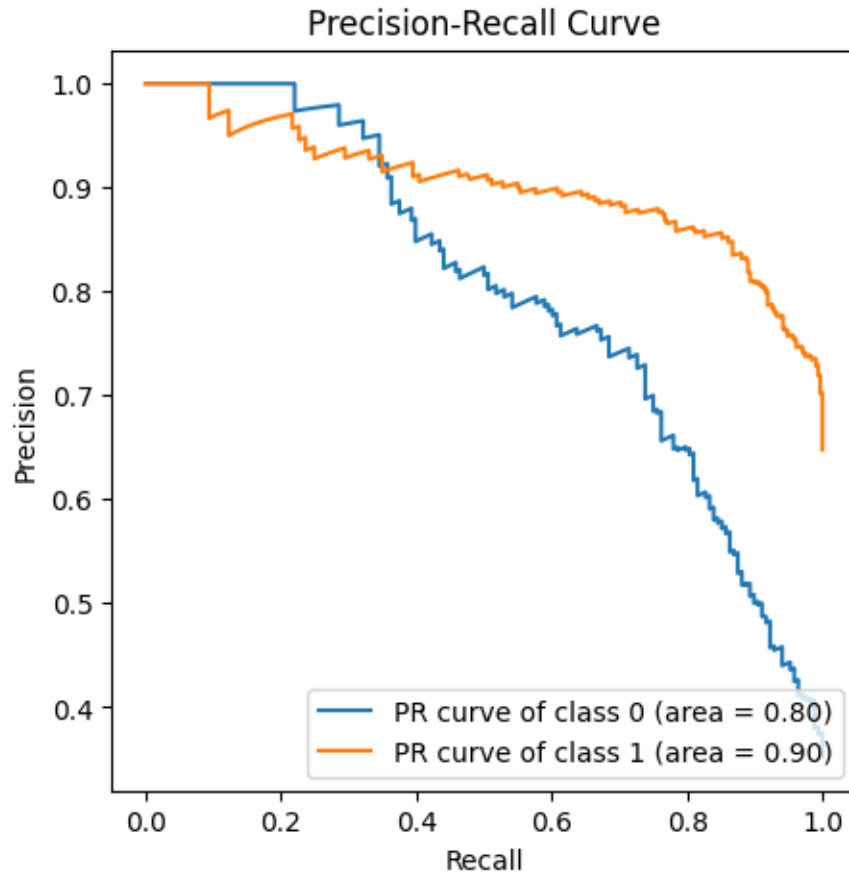
```

plot_feature_importance(xgb_random.best_estimator_, X_train.columns)
display_confusion_matrix(y_test, y_pred)
plot_multiclass_roc(xgb_random.best_estimator_, X_train, X_test, y_train,
↪y_test)
plot_pr_curve(xgb_random.best_estimator_, X_train, X_test, y_train, y_test)

```







Insight

The top 5 features as per the XGBClassifier are

--Quarterly Rating 1

--Quarterly Rating Improved --Quarterly Rating 4 --Months of Service --Quarterly Rating_Change

Both the classes 0 and 1 have a decent Area Under the ROC curve of 0.86

The Area Under the PR curve for class 0 is 0.81 and class 1 is 0.90

12 Insights

Most of the drivers are in the age group of 30 to 35

59% of the drivers are Male and remaining 41% are Female

City C20 has the maximum number of drivers

Maximum number of drivers joined in the year 2020 and in the month of July

1026 drivers have a joining designation of 1

Maximum number of drivers have a grade of 2

Majority of the drivers have a very low quarterly rating of 1

There are no drivers with quarterly rating of 5

1616 drivers have churned, which is around 68%

The median income of drivers who have churned is lesser than that of the drivers who have not

The churn rate is very less in drivers whose income has raised

The churn rate is very less in drivers whose grade has raised

The churn rate is very less in drivers whose Quarterly rating has increased

13 Recommendation

The quarterly rating has been the top contributor on deciding if a driver will churn or not. As the ratings are given by the customers to the driver, Ola should urge all customers to rate the drivers on time. Ola should provide incentives/points to the customers to encourage timely rating.

Ola should make sure that the income of deserving drivers should be increased every 6 months, if not every quarter, to encourage drivers to stay

Long service awards/bonuses should be given to drivers to keep them motivated

Special trainings should be given to drivers on how to handle different customers and different situations so that the customers always provide positive ratings